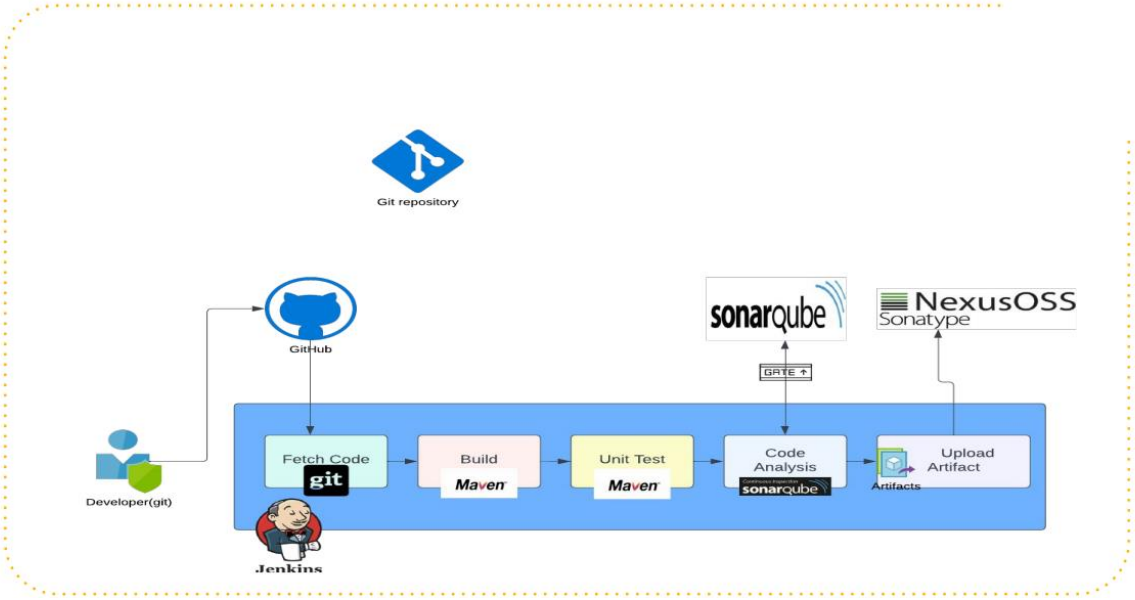
JENKINS – CONTINOUS INTEGRATION (CI)

This document provides a comprehensive overview of the Continuous Integration (CI) pipeline that will be set up and demonstrated in a series of videos. The primary goal is to understand the flow and practice the steps involved in automating the build, testing, and analysis of code.

ARCHITECTURE:



Key Tools Used:

- **Jenkins:** The central Continuous Integration tool that orchestrates the entire pipeline.
- **Git/GitHub:** A version control system and a centralized repository for storing the source code.
- **Maven:** A build automation tool used to build the Java code and run unit tests.
- **Sonarqube:** A static code analysis tool that inspects the code for bugs, vulnerabilities, and best practice violations.
- **Nexus:** A repository manager for storing and versioning the final build artifacts.

Pipeline Flow:

1. **Code Commit:**
    - A developer writes and modifies code locally.
    - After local testing, the developer pushes the changes to a centralized repository (e.g., GitHub).
2. **Triggering the Pipeline:**
    - Jenkins, configured with a Git plugin, automatically detects the new code changes in the GitHub repository.
    - Jenkins fetches the latest code using the `git` tool.

3. **Building the Code:**
    - Jenkins executes a build command using **Maven**.
    - The build process compiles the source code and generates the initial artifacts (e.g., `.jar`, `.war` files).
4. **Unit Testing:**
    - Jenkins, again using **Maven**, runs the unit tests included in the source code.
    - Maven generates reports in XML format, providing a summary of the test results.
5. **Code Analysis:**
    - The pipeline performs static code analysis to check for code quality, security vulnerabilities, and adherence to best practices.
    - **Sonarqube Scanner** and **Checkstyle** are used for this purpose.
    - Reports are generated and then uploaded to the **Sonarqube Server**.
    - **Quality Gate:** Sonarqube can be configured with a "Quality Gate," a set of rules that must be met. If the code fails to meet these standards (e.g., too many bugs or vulnerabilities), the build will be marked as a failure, and the pipeline will stop.
6. **Publishing the Artifact:**
    - If the code passes all previous steps (build, unit tests, and code analysis), the build artifact is considered "verified."
    - This verified artifact is then versioned and uploaded to a repository manager, such as **Nexus**. This ensures that only high-quality, tested artifacts are stored for future deployments.

1. Steps to Create the CI Pipeline

This document outlines the step-by-step process for setting up the Continuous Integration (CI) pipeline. This roadmap will serve as a guide throughout the setup process.

1. Server Setup

- **Jenkins Server:** Set up a new Jenkins server or use an existing one.
- **Nexus Server:** Set up a Nexus server. This will be done using a user data script (bash script) during the launch of an EC2 instance.
- **Sonarqube Server:** Set up a Sonarqube server. This will also be done using a user data script during the launch of an EC2 instance.

2. Network and Security Configuration

- **Security Group Configuration:** Configure the security groups to ensure proper communication between the Jenkins, Nexus, and Sonarqube servers. This involves allowing the necessary ports for connectivity.

3. Jenkins Plugin Installation

- Install all the required plugins in Jenkins to enable integration with other tools. This includes:
    - **Nexus Plugin**

    - **Sonarqube Plugin**
    - **Git Plugin**
    - **Maven Plugin**
    - ...and any other necessary plugins.

4. Tool Integration with Jenkins

- **Nexus Integration:** Integrate the Nexus server with Jenkins. This is a straightforward process primarily involving saving the credentials.
- **Sonarqube Integration:** Integrate the Sonarqube server with Jenkins. This process involves a few specific steps to properly configure the connection.

5. Pipeline Scripting and Execution

- **Write the Pipeline Script:** Create a pipeline script that defines the entire CI workflow, leveraging the resources and tools set up in the previous steps.
- **Execute the Pipeline:** Run the pipeline script to execute the complete automated process of fetching, building, testing, and publishing the code.

6. Notifications

- **Set up Notifications:** Configure automatic notifications (e.g., email or other communication channels) to be sent whenever a pipeline job fails. This ensures that the team is immediately aware of any issues in the CI process.

---

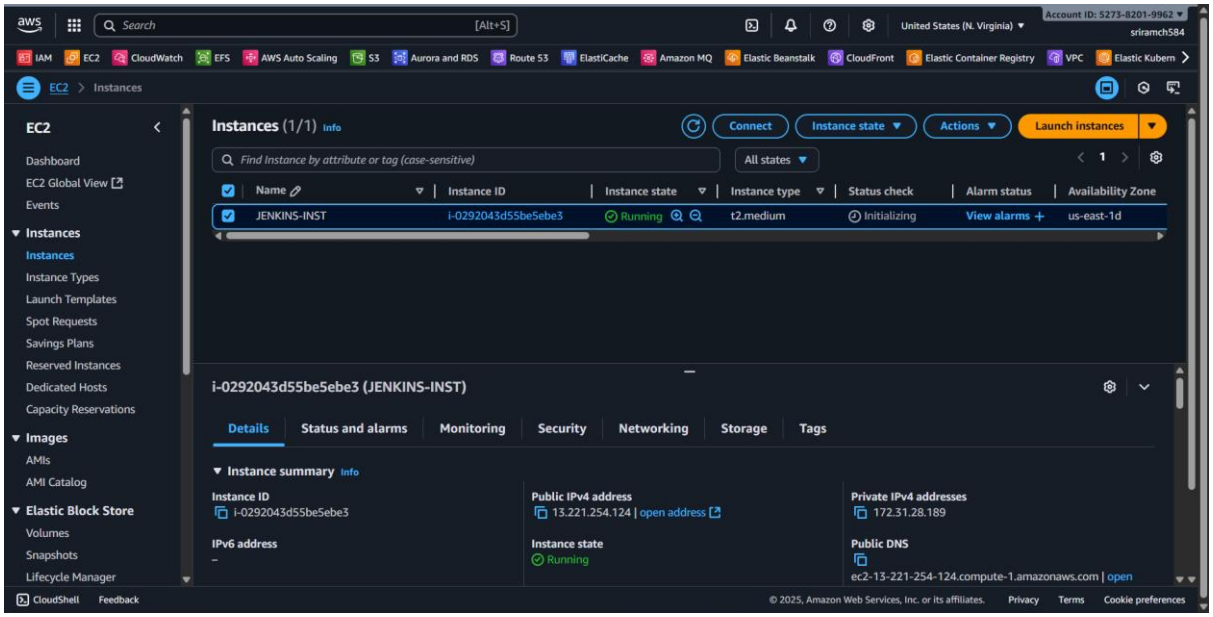
2. Setting Up Jenkins, Nexus, and Sonarqube ⚙

This guide outlines the steps for setting up Nexus and Sonarqube servers on AWS EC2 instances and configuring the security groups for communication with a pre-existing Jenkins server.

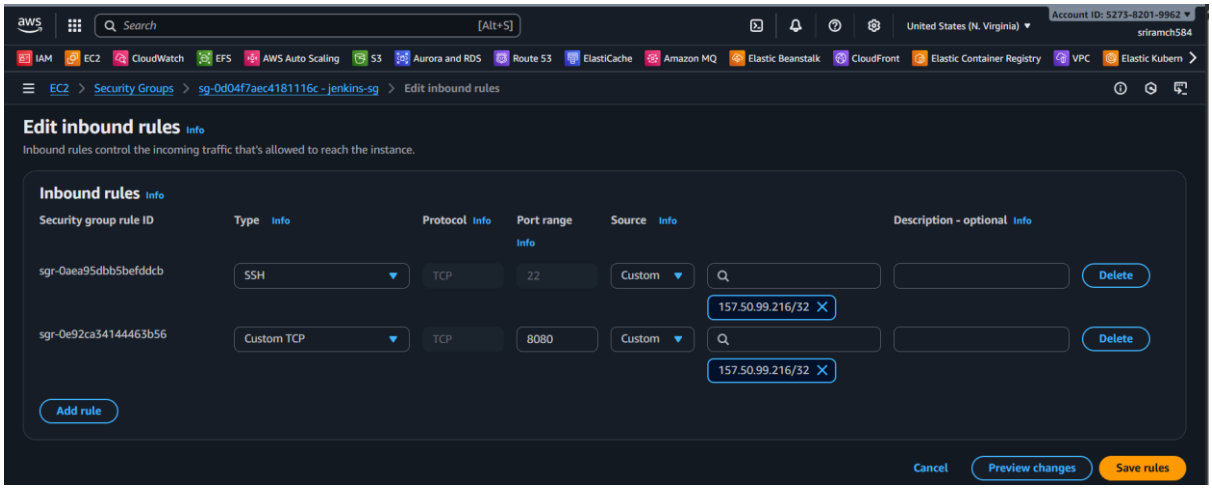
Jenkins Setup

While the video assumes you already have a Jenkins server, here's a brief overview of how it's typically set up on an EC2 instance:

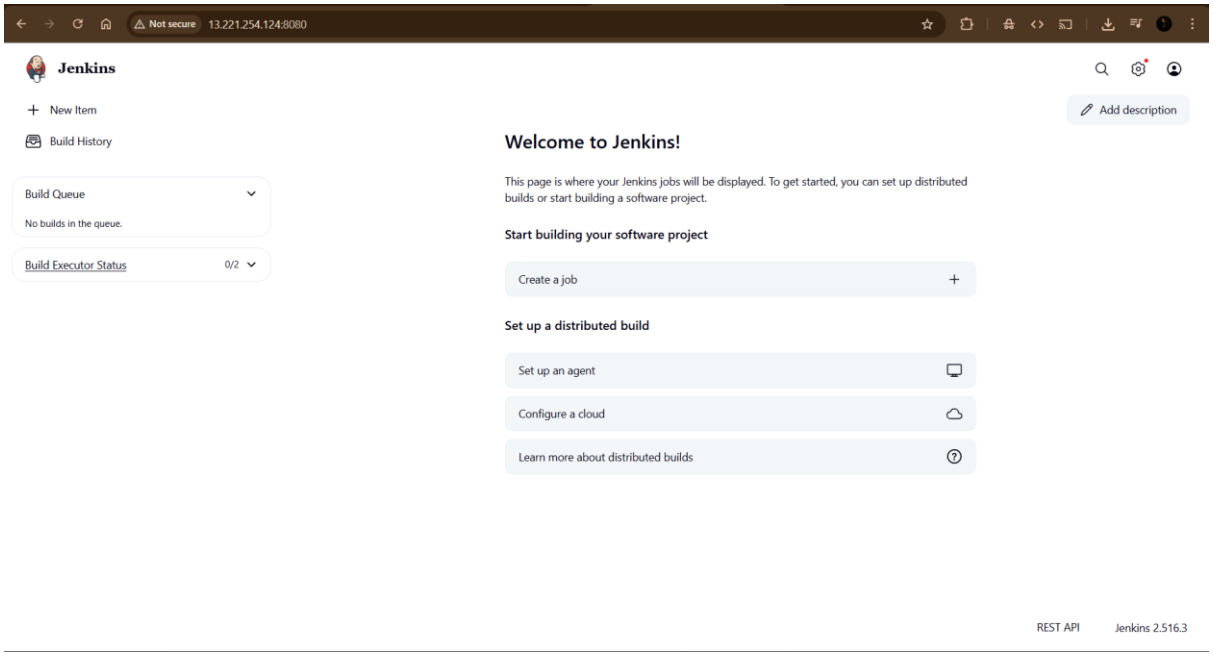
- **Launch EC2 Instance:** Launch an EC2 instance (e.g., T2.medium) with a compatible OS like Amazon Linux 2 or Ubuntu.



- **Install Java:** Install a compatible Java Development Kit (JDK) on the instance.
- **Install Jenkins:** Add the Jenkins repository and install the Jenkins package using your OS's package manager (`yum` or `apt`).
- **Start Service:** Start and enable the Jenkins service using `systemctl start jenkins`.
- **Security Group:** Create a security group for Jenkins (e.g., Jenkins-SG) that allows inbound traffic on port **8080** for the Jenkins web UI and port **22** for SSH.

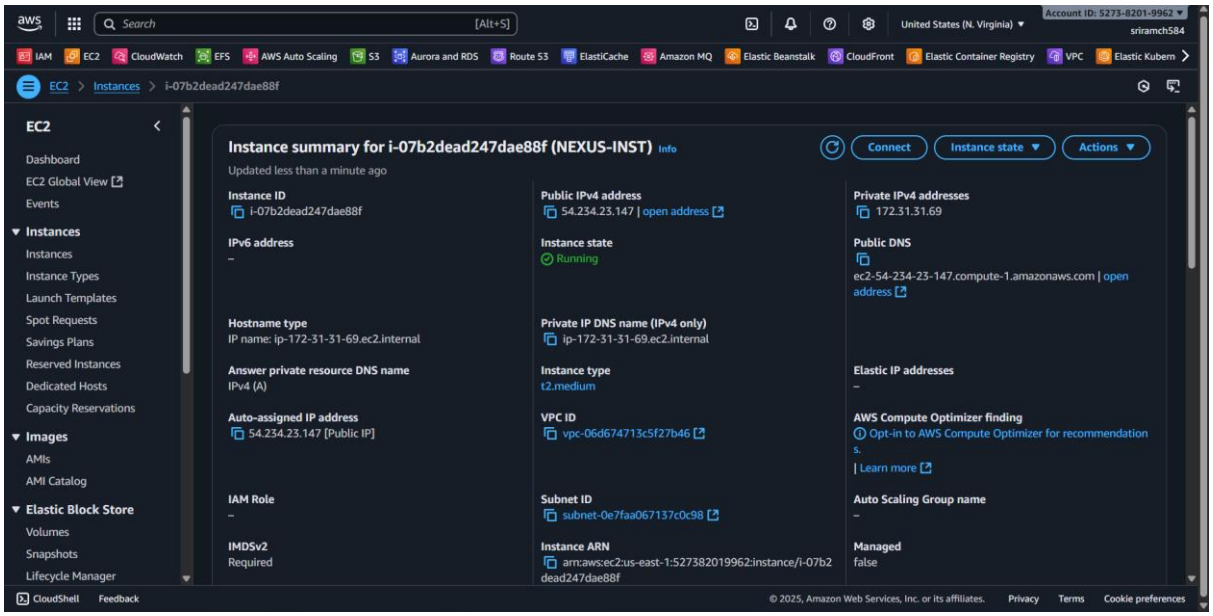


Once Jenkins is running, you can access it via a web browser using the instance's public IP address and port 8080 (`http://<Jenkins_Public_IP>:8080`).



Nexus Server Setup

The Nexus server is set up on an Amazon Linux 2023 EC2 instance using a UserData bash script. The script automates the entire process, including:

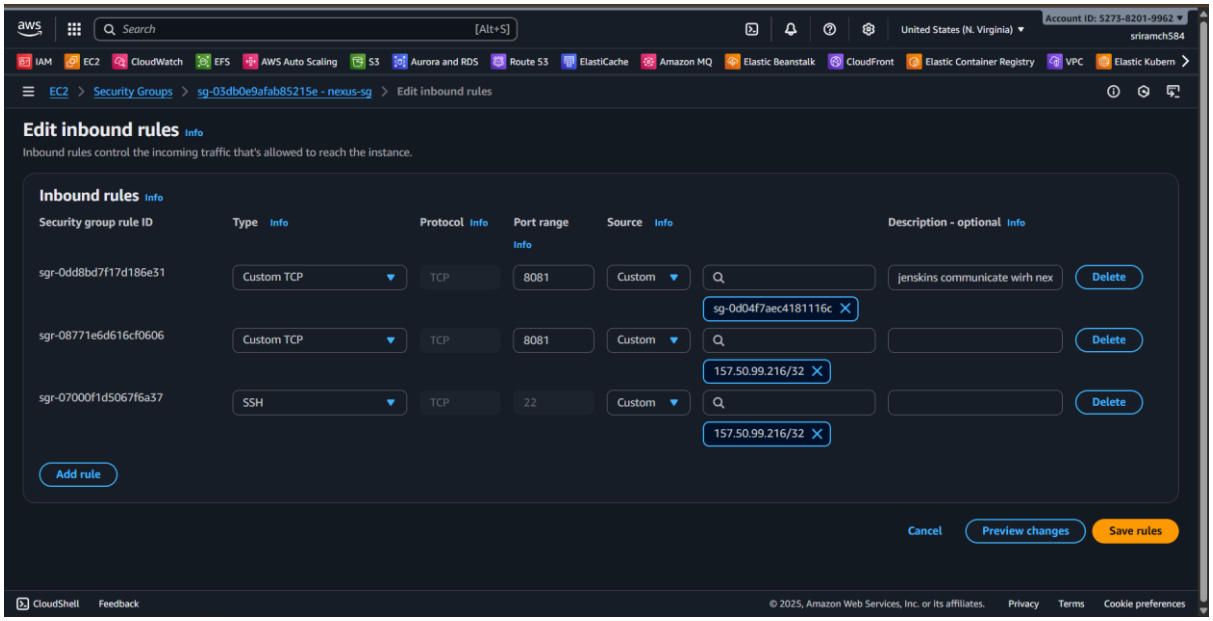


- Installing **JDK 17**.
- Downloading and extracting the Nexus binary.
- Creating a dedicated `nexus` user.
- Creating a systemd service file to manage the Nexus service.
- Starting and enabling the Nexus service.

Security Group Configuration:

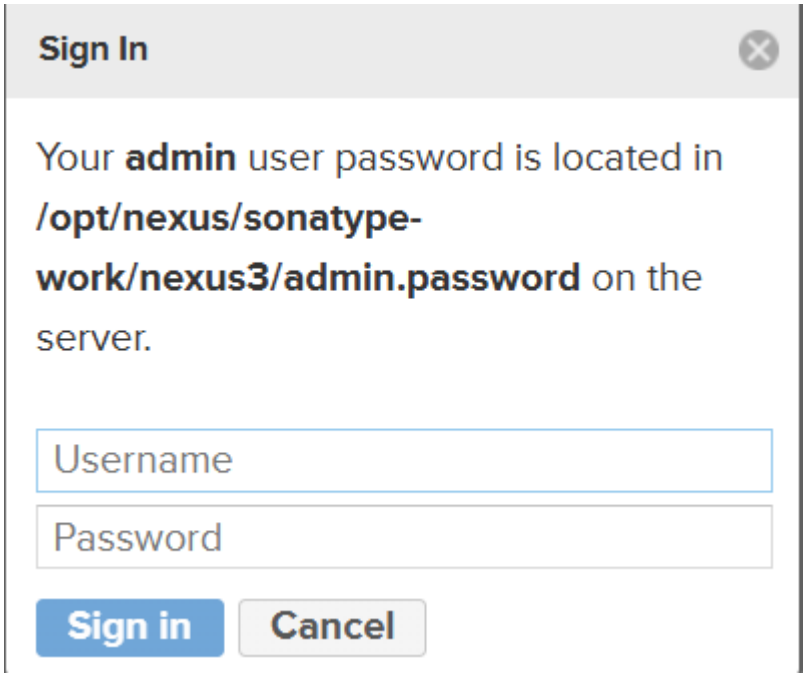
- A new security group, **Nexus-SG**, is created.

- **Inbound rules** are configured to allow access to:
  - **Port 22 (SSH)** from your IP.
  - **Port 8081 (Nexus web UI)** from your IP and from the **Jenkins Security Group**. This allows Jenkins to upload artifacts to Nexus.

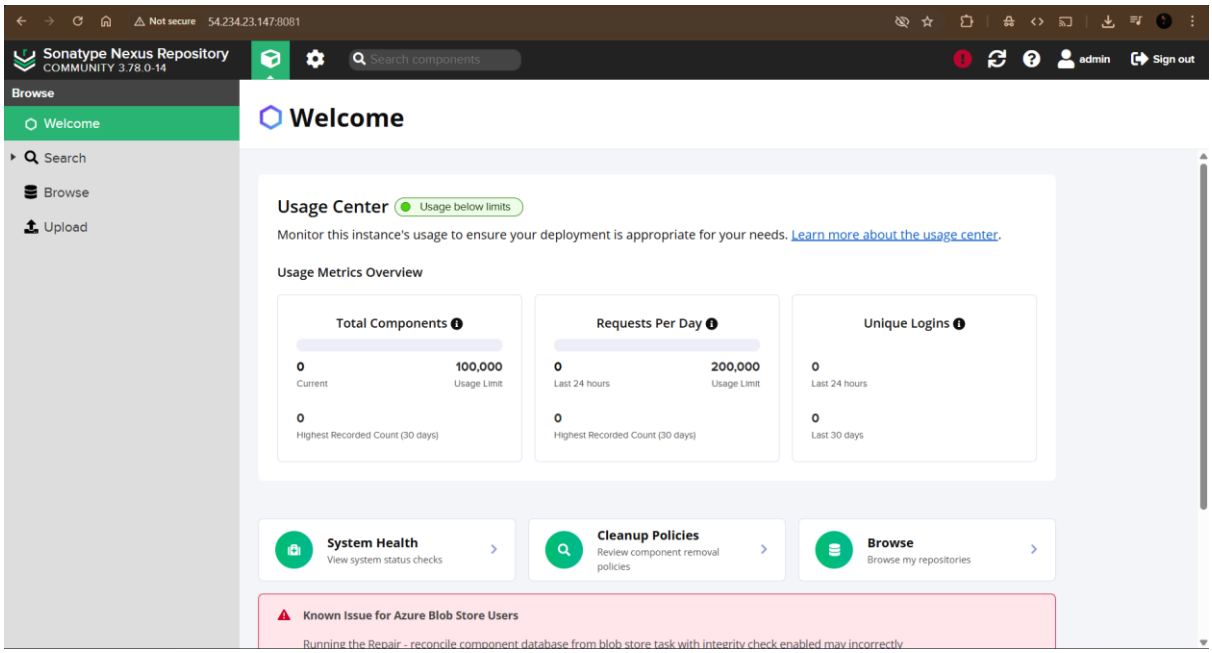


Accessing Nexus:

- After the instance is launched and the script completes, you can access the Nexus web UI by navigating to `http://<Nexus_Public_IP>:8081` in a browser.
- The initial password can be found by ssh into the instance and running `cat /opt/nexus/sonatype-work/nexus3/admin.password`.



- You will be prompted to change the password and can then set up anonymous access to the repositories.

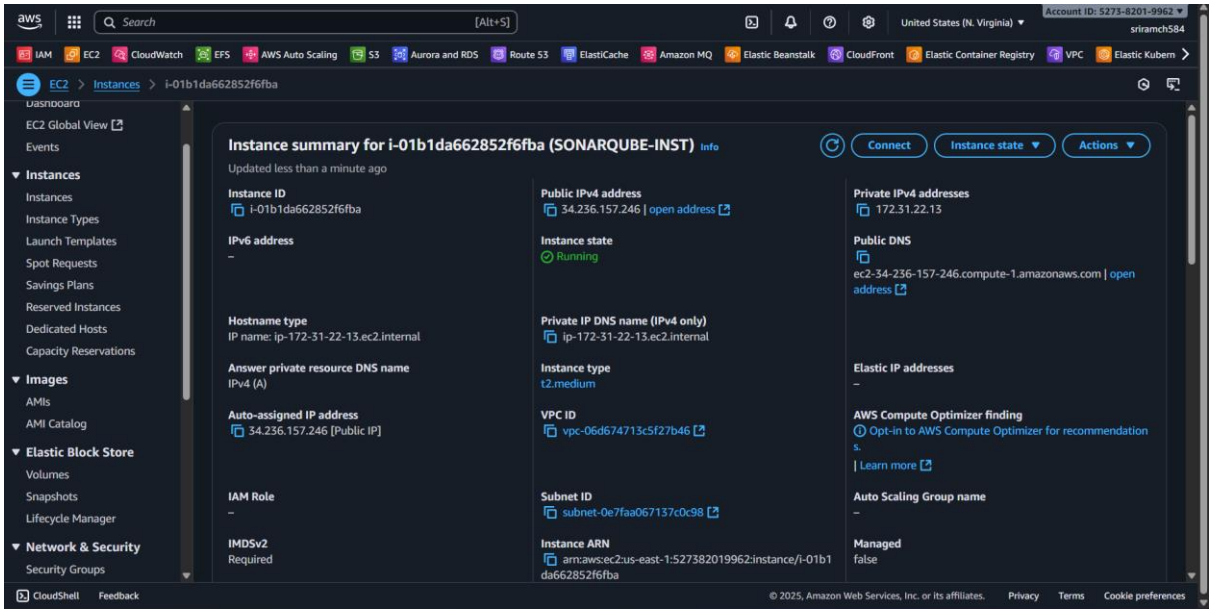


Sonarqube Server Setup

The Sonarqube server is set up on an Ubuntu 24.04 EC2 instance. The UserData script for this setup is more complex as it configures **three services on a single machine**: Sonarqube, PostgreSQL database, and Nginx as a frontend proxy.

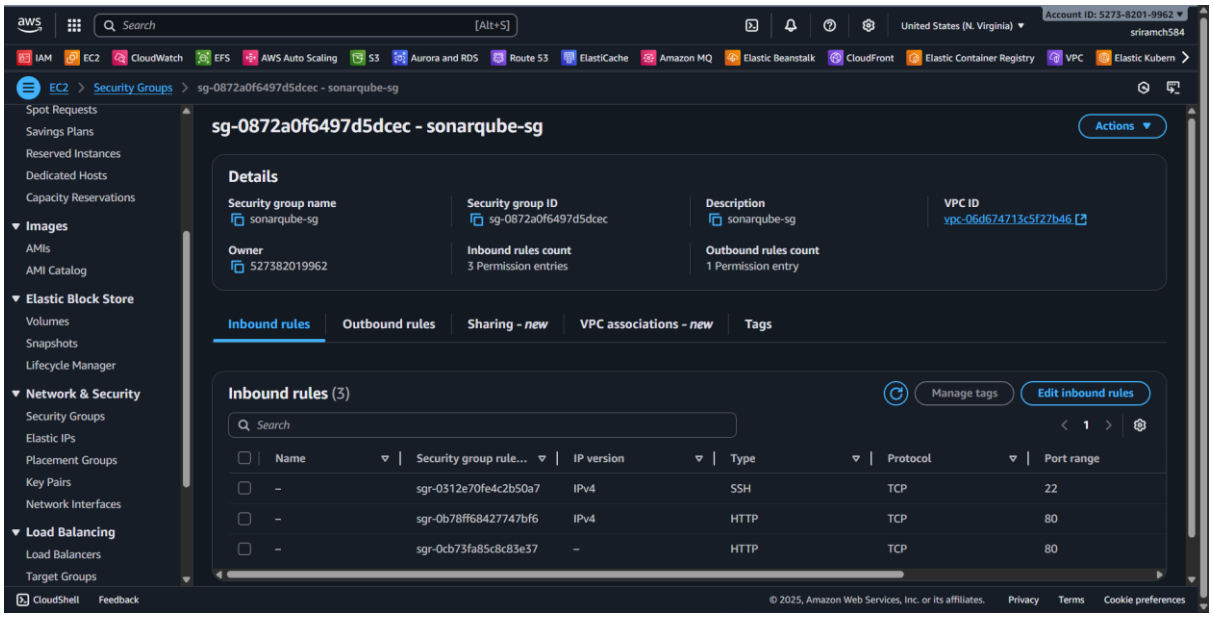
Key Script Details:

- **Operating System Level Changes:** The script includes commands to modify system files like `/etc/sysctl.conf` and `/etc/security/limits.conf` to increase file descriptor limits. These changes are necessary to provide adequate resources for Sonarqube and PostgreSQL. A reboot is performed after these changes.
- **PostgreSQL Setup:** The script installs PostgreSQL, creates a `sonar` database, and grants privileges to a `sonar` user with a password of `admin123`.
- **Sonarqube Installation:** Similar to Nexus, it downloads, extracts, and configures the Sonarqube binary. The `sonar.properties` file is updated to connect to the PostgreSQL database.
- **Nginx Configuration:** Nginx is installed as a frontend service, listening on **Port 80**. It is configured to forward requests to Sonarqube, which runs on **Port 9000**.



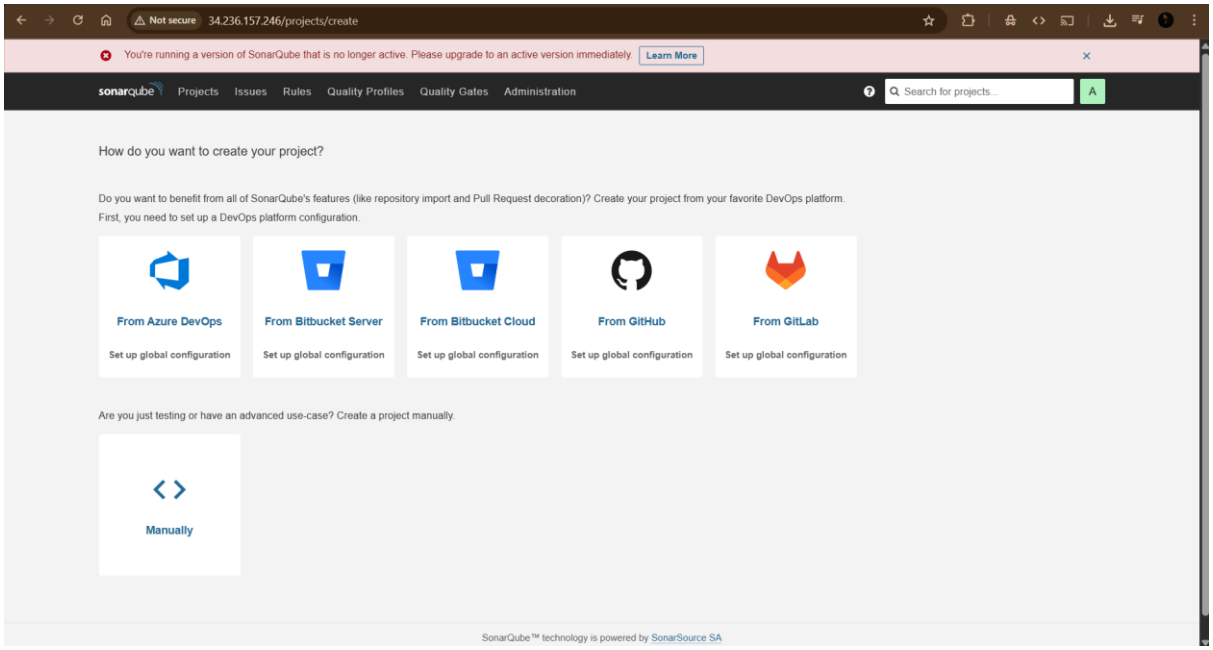
Security Group Configuration:

- A new security group, **Sonar-SG**, is created.
- **Inbound rules** are set to allow access to:
    - **Port 22 (SSH)** from your IP.
    - **Port 80 (Nginx/SonarQube UI)** from your IP and from the **Jenkins Security Group**. This allows Jenkins to upload code analysis results.



Accessing Sonarqube:

- After the instance is launched and the script completes, you can access the Sonarqube web UI by navigating to `http://<Sonarqube_Public_IP>` in a browser.
- The default login credentials are `admin` for both the username and password.



Inter-Server Communication

- To ensure the entire CI pipeline functions correctly, bidirectional communication must be allowed between the servers.
- **Jenkins** connects to **Nexus** on **Port 8081** to upload artifacts.
- **Jenkins** connects to **Sonarqube** on **Port 80** to upload analysis results.
- **Sonarqube** contacts **Jenkins** on **Port 8080** to send back the status of the quality gate check.
- This last connection requires an additional **inbound rule** in the **Jenkins Security Group** to allow **Port 8080** traffic from the **Sonar-SG**.

3. Plugin Installation in Jenkins

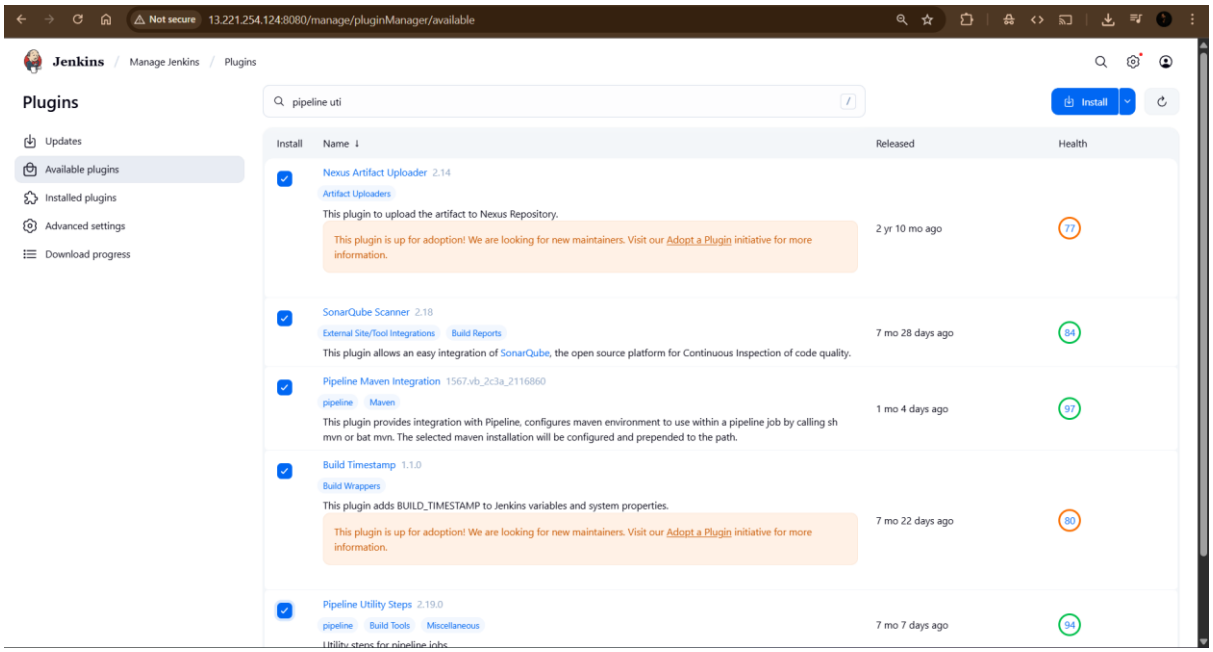
The following plugins will be installed:

- **Nexus Artifact Uploader:** This plugin facilitates the uploading of build artifacts from Jenkins to the Nexus repository.
- **Sonarqube Scanner:** This plugin is essential for integrating Jenkins with Sonarqube, allowing Jenkins to initiate code analysis and send the results to the Sonarqube server.
- **Build Timestamp:** This plugin is used for versioning the build artifacts, ensuring each artifact has a unique and identifiable timestamp.
- **Pipeline Maven Integration:** This plugin is crucial for running Maven builds within a Jenkins pipeline script.
- **Pipeline Utility Steps:** This plugin provides a set of useful steps for a pipeline, such as reading and writing files.

The **Git plugin** is already installed by default, so it does not need to be manually installed.

Installation Steps

1. Navigate to **Manage Jenkins** in the Jenkins dashboard.
2. Click on **Manage Plugins**.
3. Go to the **Available** tab.
4. Search for each plugin by its name:
    - `Nexus Artifact Uploader`
    - `Sonarqube Scanner`
    - `Build Timestamp` (search for `timestamp`)
    - `Pipeline Maven Integration`
    - `Pipeline Utility Steps`
5. Select the checkboxes next to each plugin.
6. Click **Install without restart** to install the selected plugins.



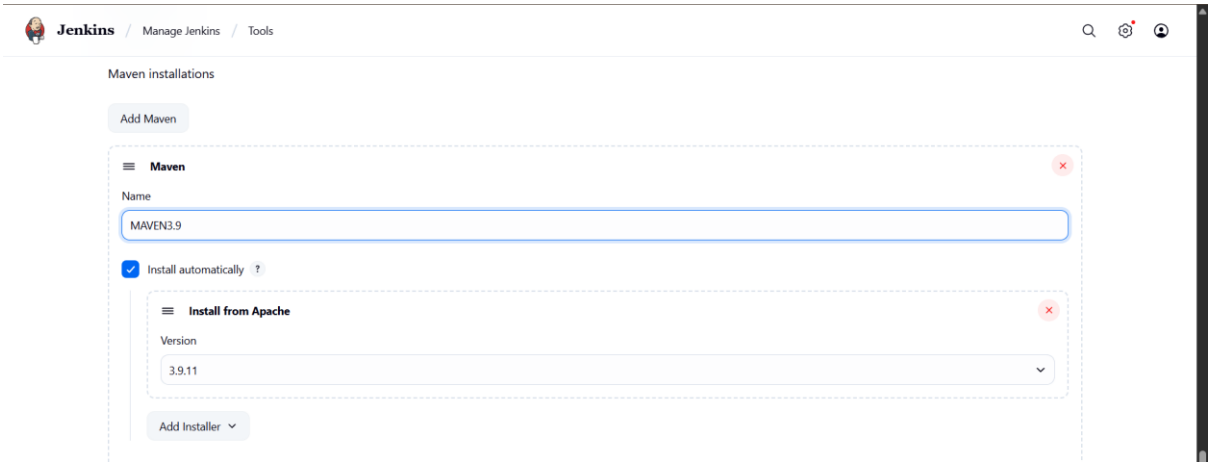
4. Building the Artifact with Pipeline as a Code

The Jenkinsfile is a script that defines the entire CI/CD pipeline, including stages, steps, and tool configurations.

Before building the arifact

- We have to add the JDK in the tools: Manage Jenkins→ Tools→JDK installations.

- Next also add the maven version in tools: Manage Jenkins→Tools→Maven installations



Creating a Basic Jenkinsfile

A basic Jenkinsfile automates the fetching of source code, building, and running unit tests.

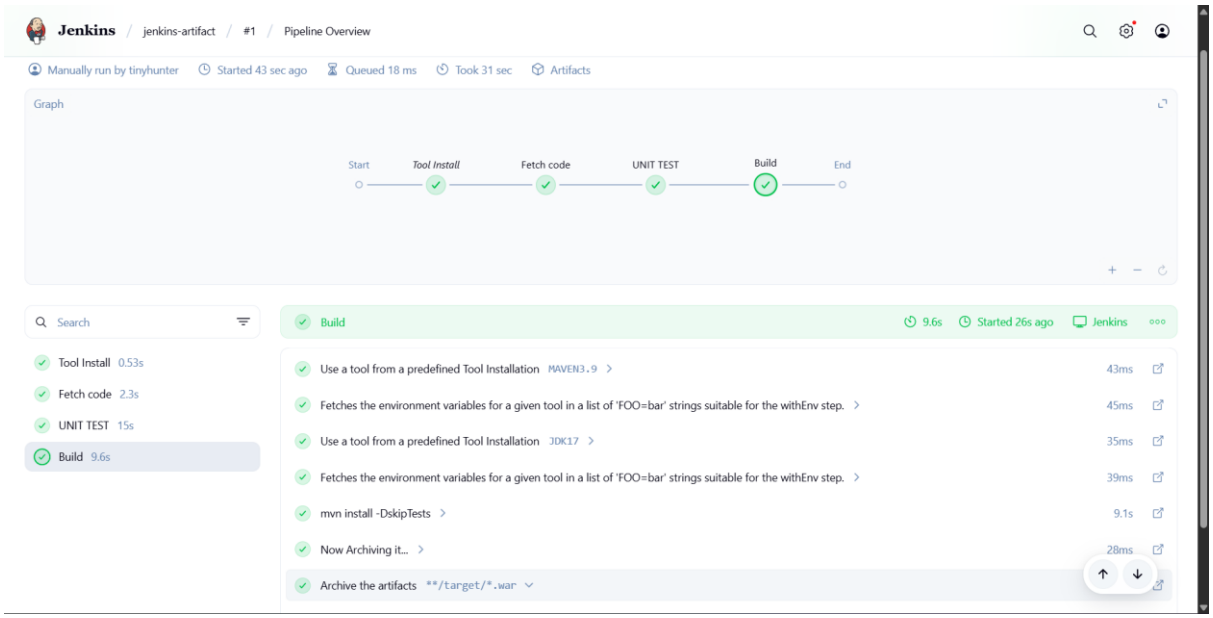
Sample Jenkinsfile

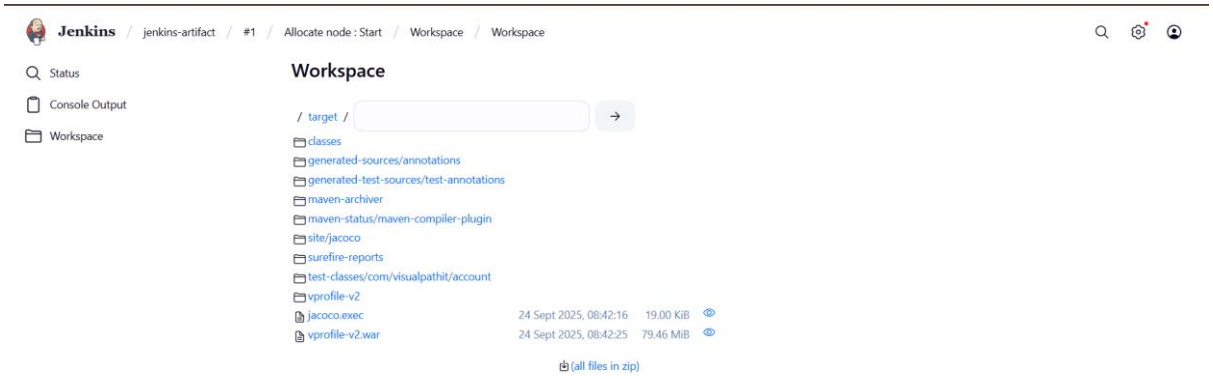
The following code demonstrates a simple Jenkinsfile with three stages: `Fetch Code`, `Unit Test`, and `Build`.

Executing the Pipeline

1. In Jenkins, go to **New Item** and create a new **Pipeline** job.
2. Give it a name (e.g., `Jenkins-artifacte`).
3. In the job configuration, select **Pipeline script** and paste the entire Jenkinsfile content into the script text area.

4. Save the job and click **Build Now**. Jenkins will then execute the pipeline, and you can monitor the progress of each stage from the pipeline view.





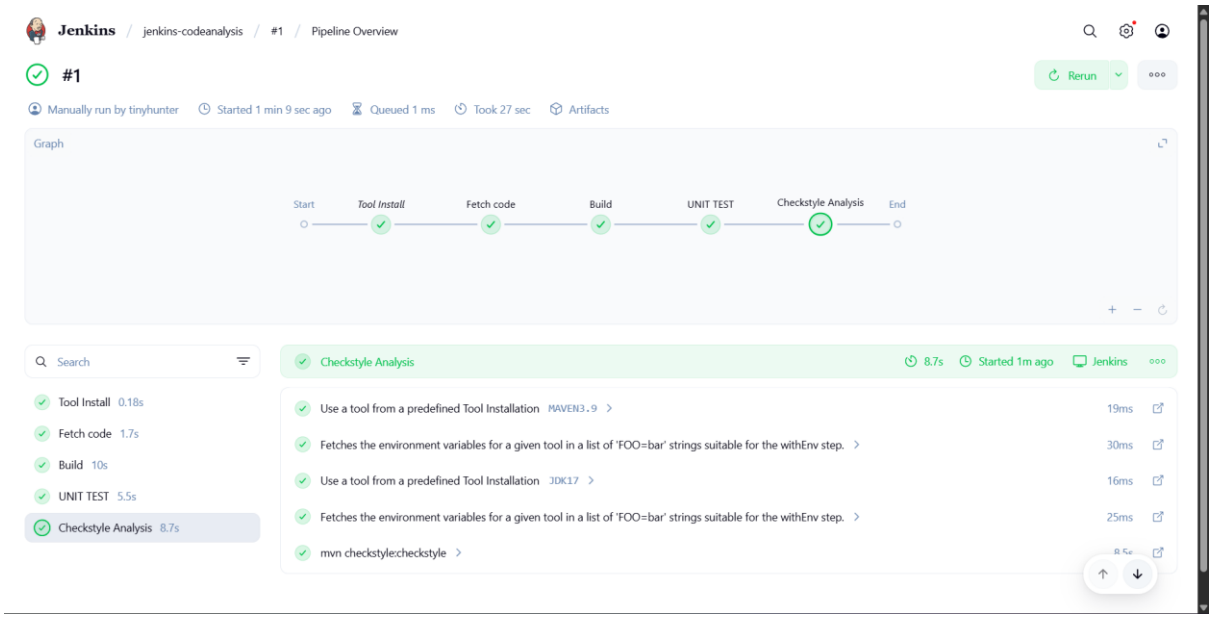
5. Code Analysis and Sonarqube Integration

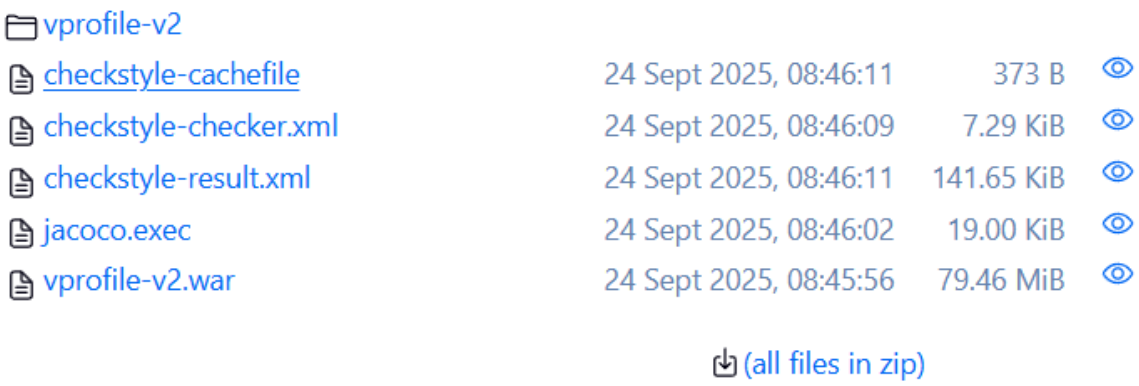
What is Code Analysis?

Code analysis is a type of automated software testing that inspects source code to identify potential issues without executing the program. It's a crucial part of the development lifecycle that helps to improve code quality.

- **Best Practices:** It checks if the code adheres to a set of best practices and coding standards. This helps in maintaining consistent and readable code across a project.
- **Vulnerabilities:** It scans for security vulnerabilities, such as the **OWASP Top 10**, which are common weaknesses that could be exploited by attackers.
- **Bugs:** It can detect bugs, logical flaws, and other functional errors that might not be immediately apparent during regular testing.

Code analysis tools like **Checkstyle** and **Sonarqube** are used to perform these checks and generate reports on the code's health.

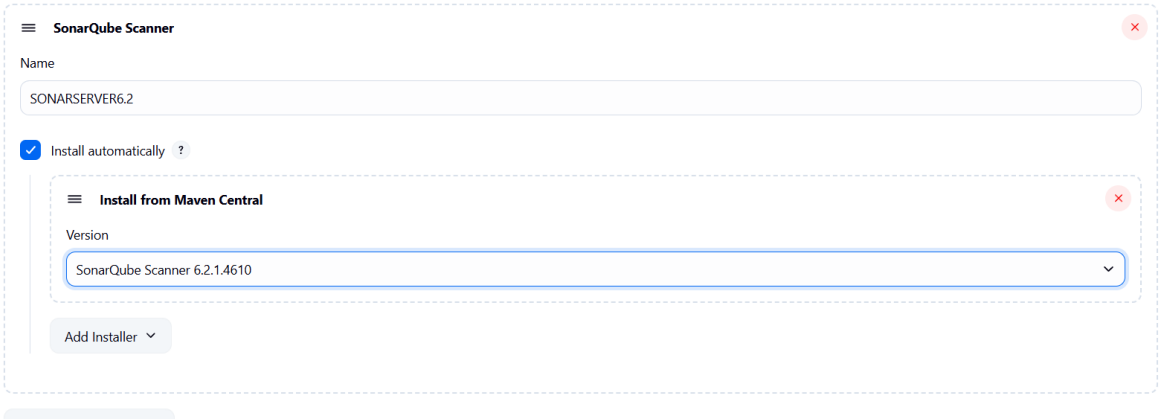




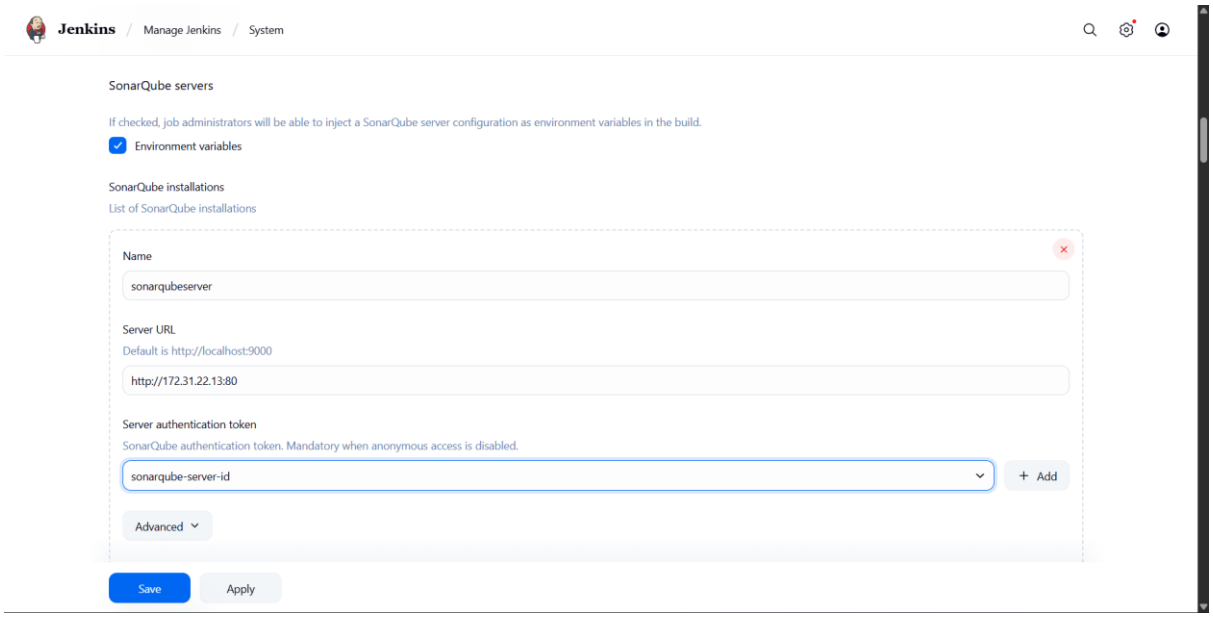
Integrating Sonarqube with Jenkins

To perform code analysis within the Jenkins CI pipeline, two key integrations are required:

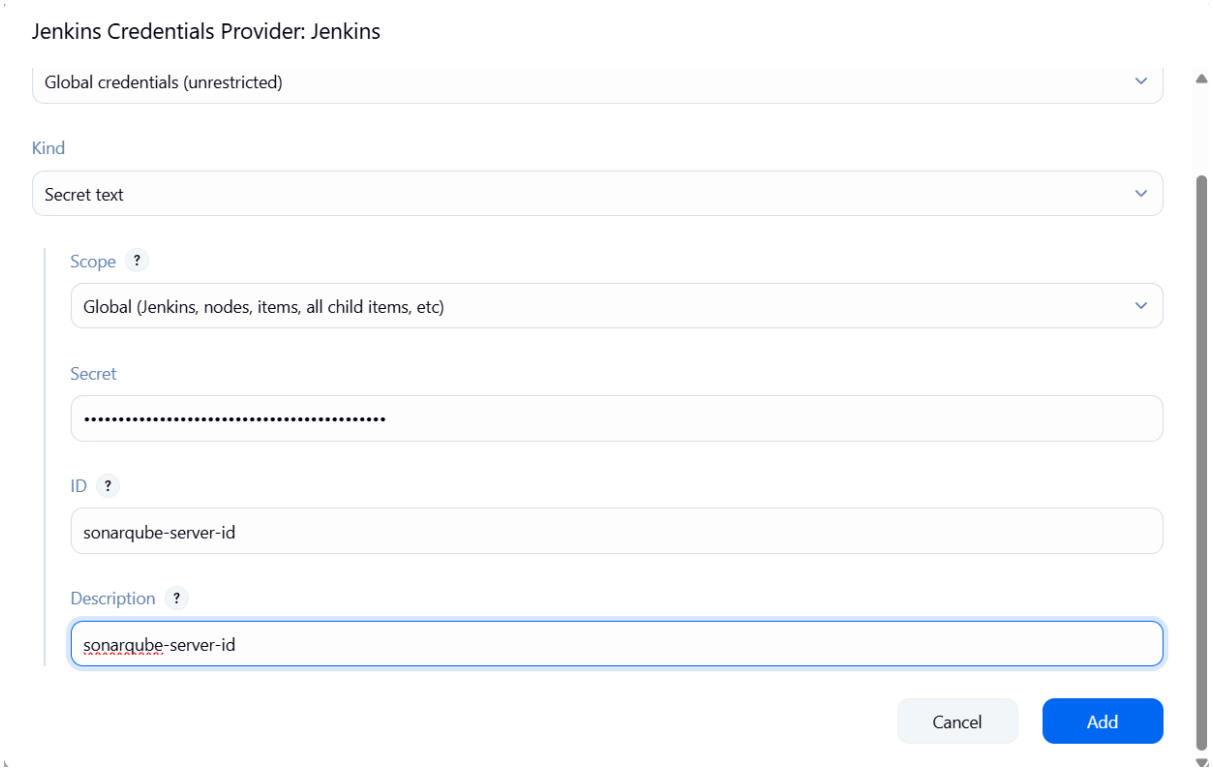
1. **Installing the Sonarqube Scanner Tool:** Jenkins needs the Sonarqube Scanner tool to analyze the source code.
    - Navigate to **Manage Jenkins** -> **Tools**.
    - Scroll down to **SonarQube Scanner installations**.
    - Click **Add SonarQube Scanner**.
    - Provide a name (e.g., `Sonar-6.2`) and select the recommended version. This name will be referenced in the Jenkinsfile.
    - Save the configuration.



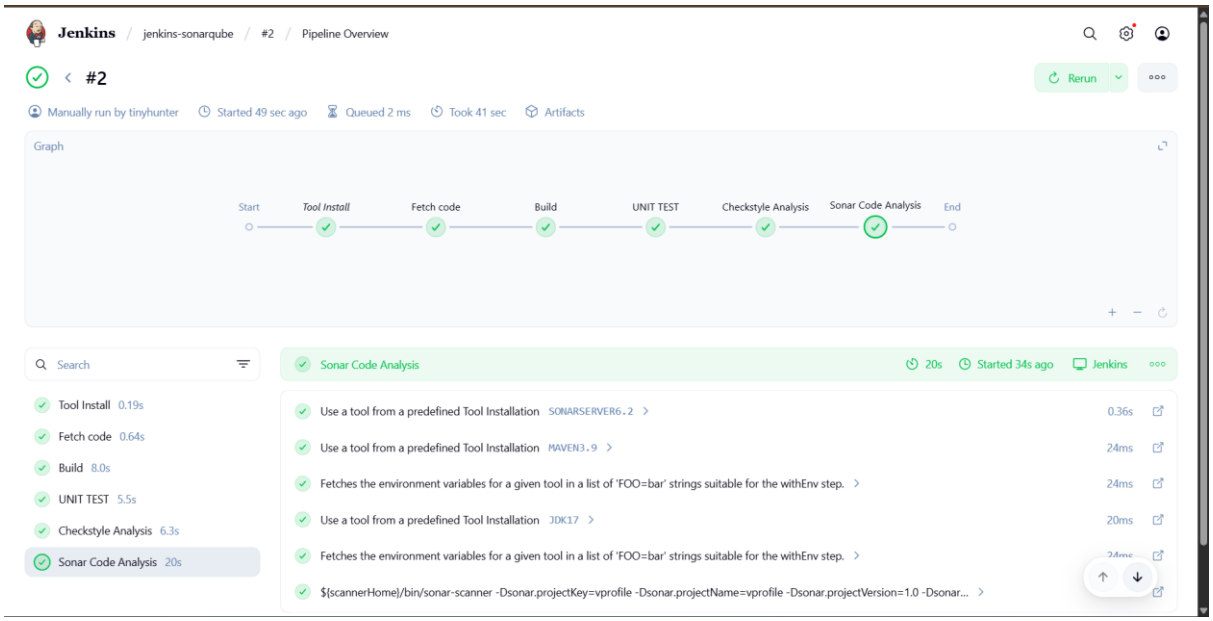
2. **Configuring Sonarqube Server Details:** Jenkins needs to know where to send the analysis results. This involves providing the Sonarqube server's details and an authentication token.
    - In Jenkins, go to **Manage Jenkins** -> **System**.
    - Scroll down to **SonarQube servers** and click **Add SonarQube**.
    - Give it a name (e.g., `Sonar-Server`) and provide the Sonarqube server's private IP address in the URL field (e.g., `http://<Sonar Private IP>`).



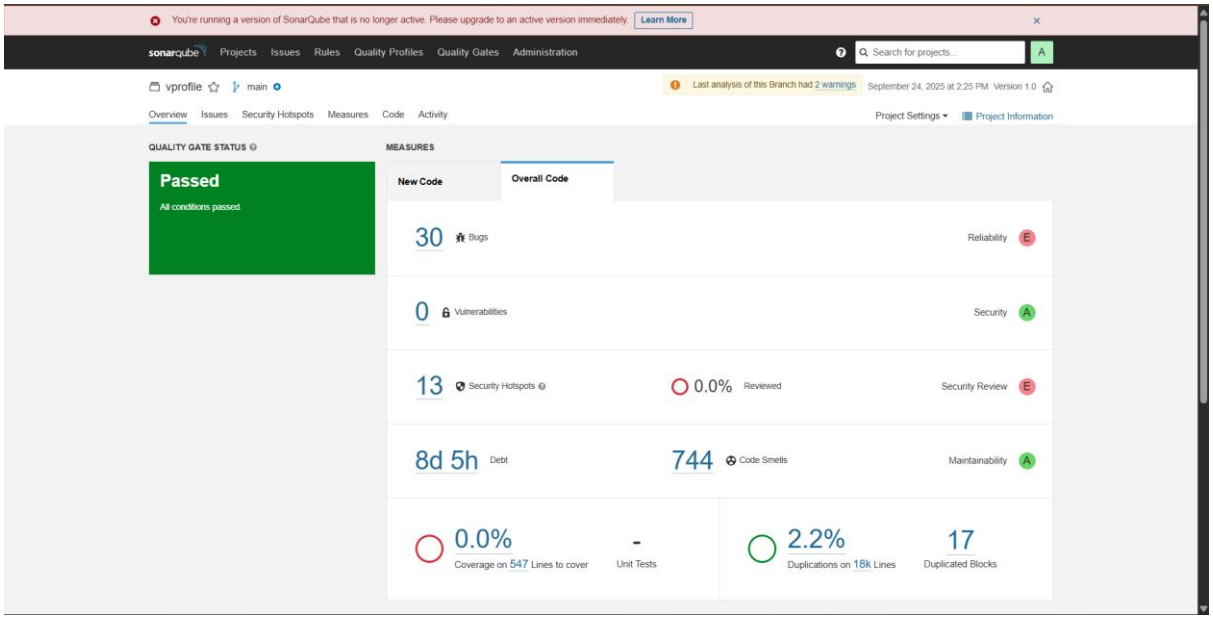
    - Using a private IP is more secure as it keeps traffic within the same network.
    - For authentication, a **token** is required.
        - Log in to the Sonarqube web UI.
        - Click on your profile icon -> **My Account** -> **Security**.
        - Generate a new token (e.g., `Jenkins`). **Copy this token immediately** as it is a one-time secret.
    - Back in Jenkins, click **Add** under **Server authentication token**.
    - Select **Secret text**, paste the token, give it an ID (e.g., `Sonar-Token`), and click **Add**.



- Select the newly created token from the dropdown.
- Ensure the security group of the Sonarqube server allows inbound traffic from the Jenkins security group on **port 80**.
- Save the changes.
- After that run the build, in this I am creating the job name “Jenkins-sonarqube”

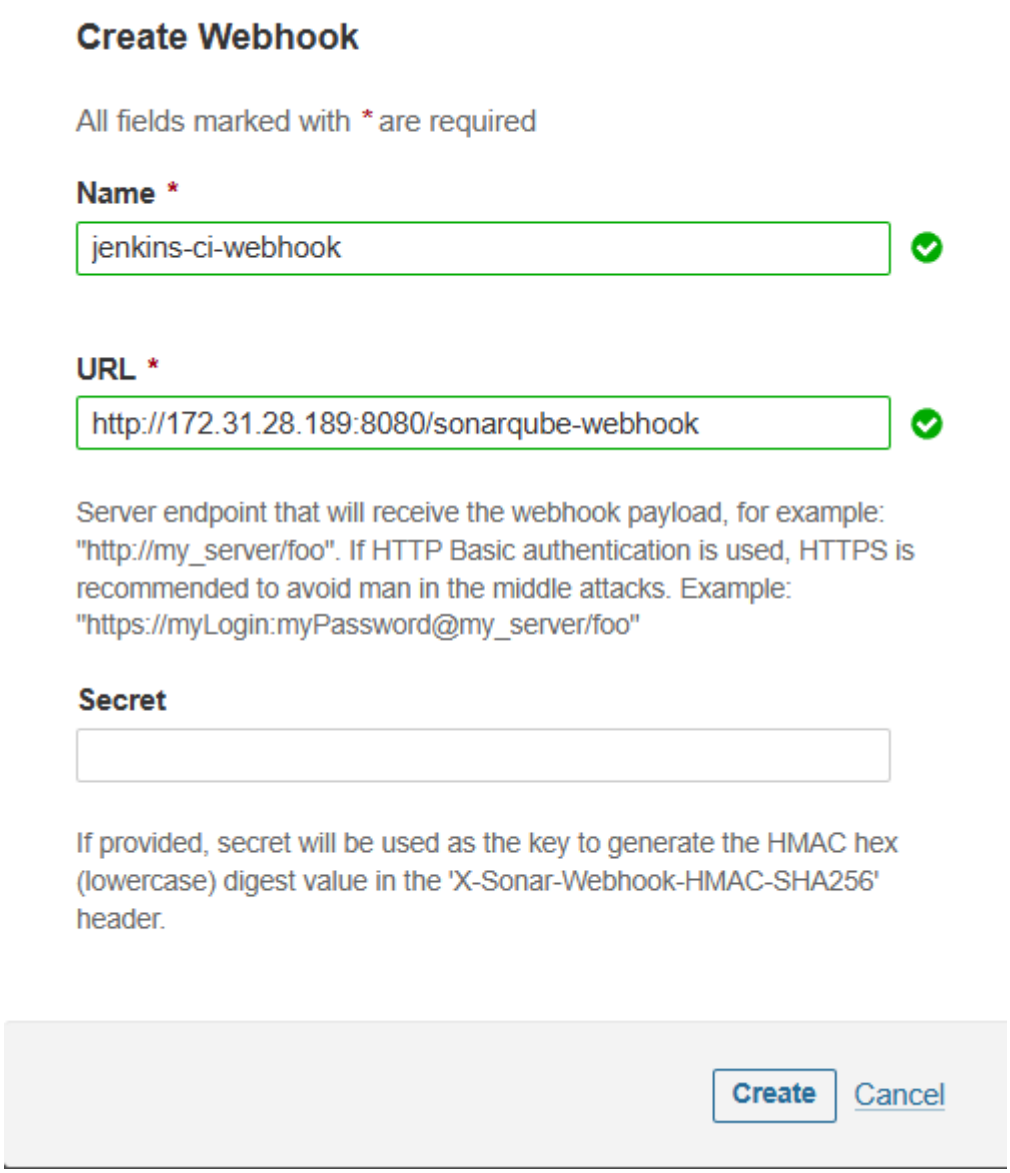


- Check the sonarqube server and check the details.

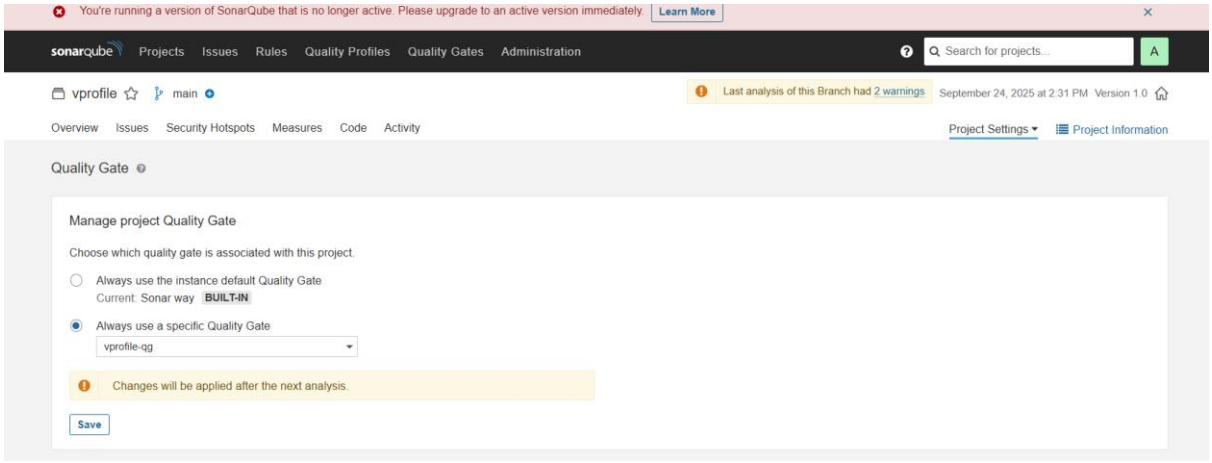

You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately.
Learn More
sonarqube
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
Search for projects...
A
vprofile
main
Last analysis of this Branch had 2 warnings
September 24, 2025 at 2:25 PM Version 1.0
Overview
Issues
Security Hotspots
Measures
Code
Activity
Project Settings
Project Information
QUALITY GATE STATUS
Passed
All conditions passed.
MEASURES
New Code
Overall Code
30
Bugs
Reliability
E
0
Vulnerabilities
Security
A
13
Security Hotspots
0.0%
Reviewed
Security Review
E
8d 5h
Debt
744
Code Smells
Maintainability
A
0.0%
Coverage on 547 Lines to cover
-
Unit Tests
2.2%
Duplications on 18k Lines
17
Duplicated Blocks

6. Quality Gates

- For creating the quality gates that can be classified by using the webhooks.
- In that provide the name & the private ip of the Jenkins and add the url after the ip address such as the sonarqube-webhook.



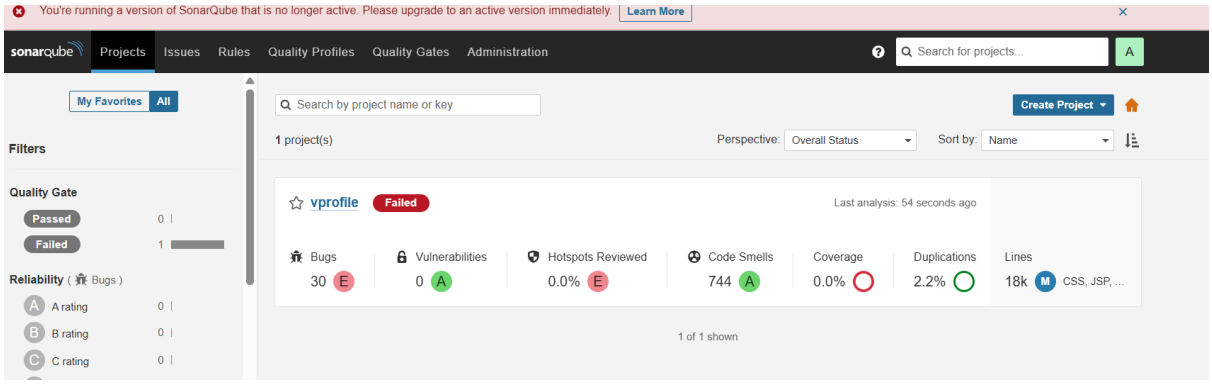
- Here I used an customize quality which it mainly focus on the (bugs < 10) And applying the quality gate.



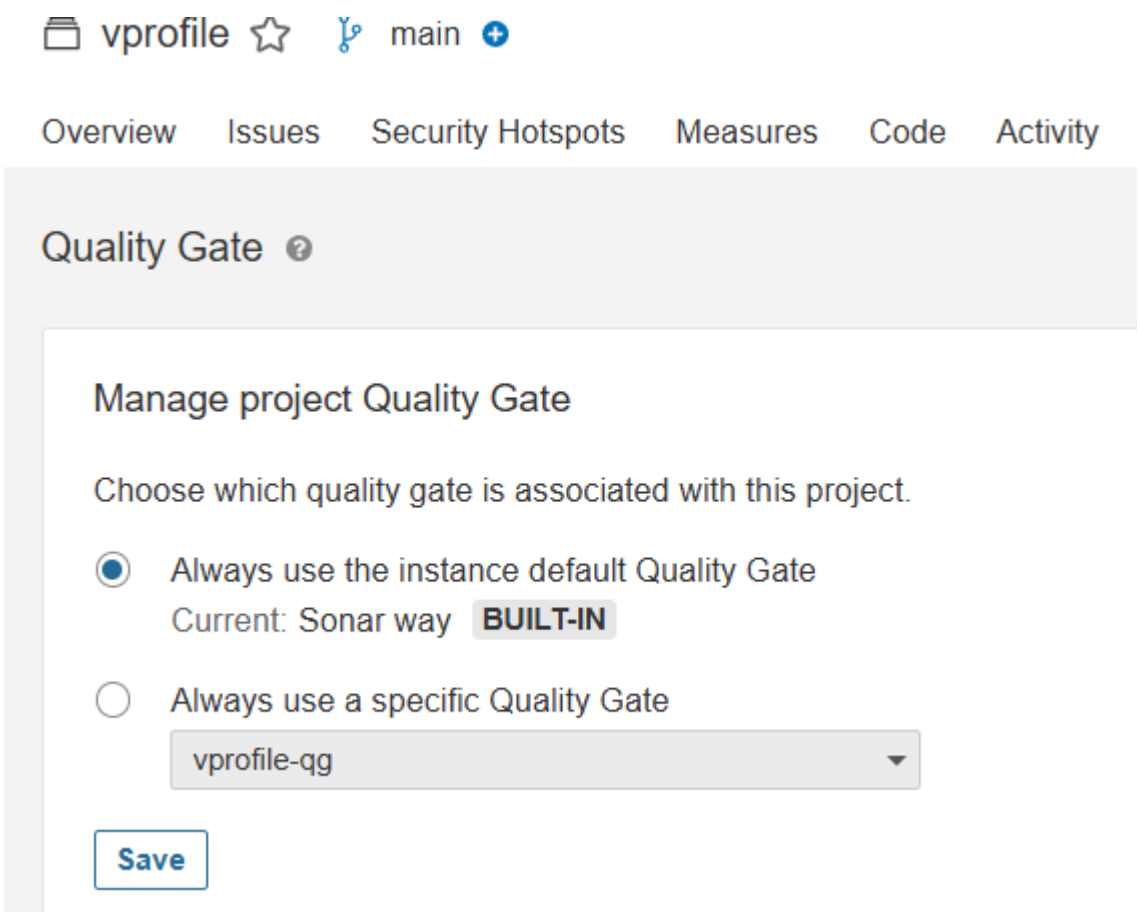
- When we run the job it fails.

```
Checking status of SonarQube task 'AZl6_awAMf7E7y2obFna' on server 'sonarqubeserver'
SonarQube task 'AZl6_awAMf7E7y2obFna' status is 'IN_PROGRESS'
SonarQube task 'AZl6_awAMf7E7y2obFna' status is 'SUCCESS'
SonarQube task 'AZl6_awAMf7E7y2obFna' completed. Quality gate is 'ERROR'
[Pipeline] }
[Pipeline] // timeout
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: Pipeline aborted due to quality gate failure: ERROR
Finished: FAILURE
```

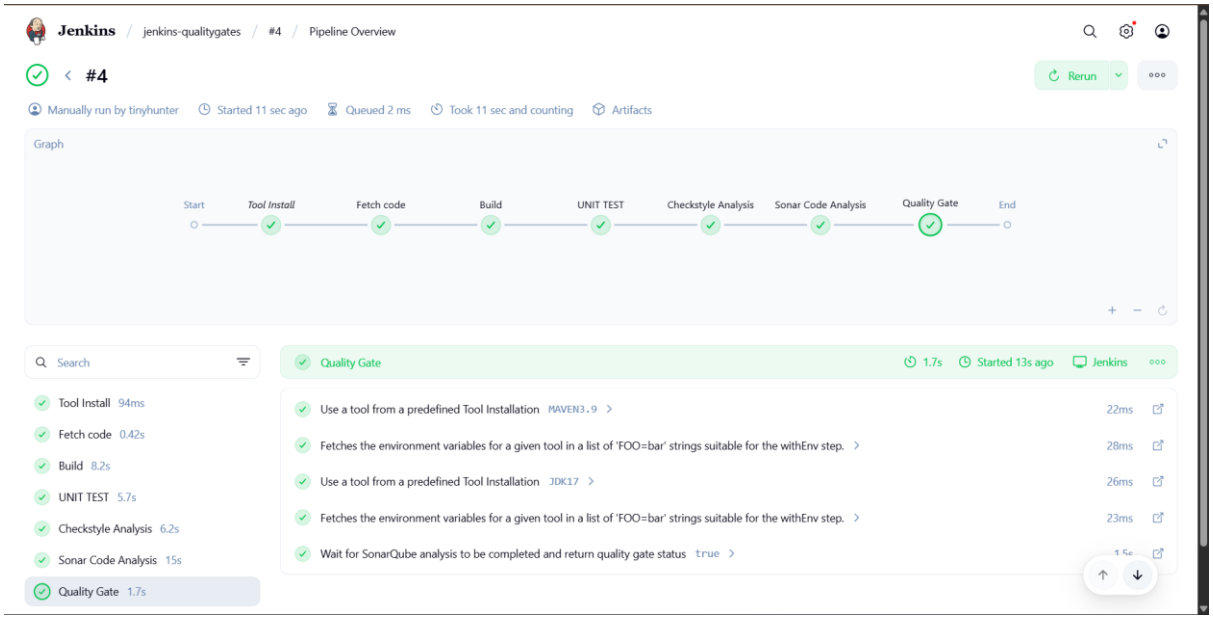
- Our project can having the 30 bugs that’s why it failed.



- When we use the default quality gate,

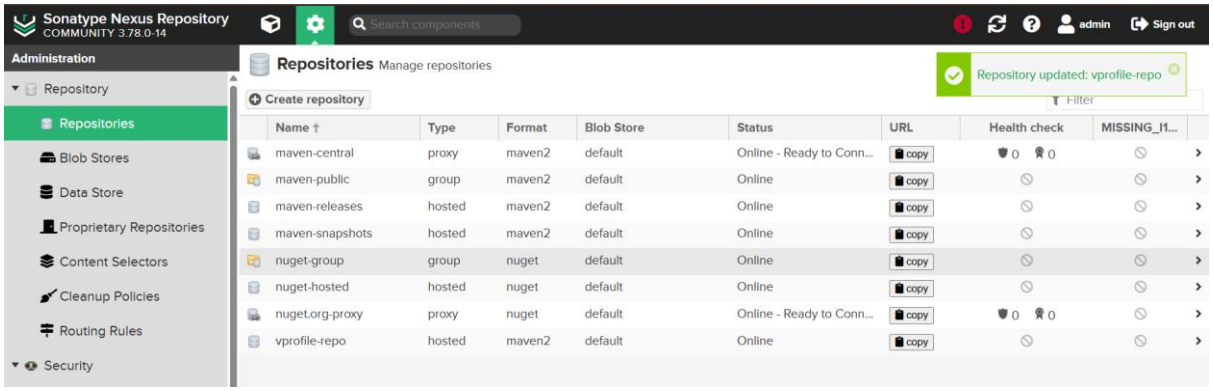


- Then the jobs success.



7. Setting Up a Nexus Repository

1. **Access the Nexus Server:** Ensure the Nexus EC2 instance is running and access its web UI using the public IP and port `8081` (`http://<Nexus_Public_IP>:8081`).
2. **Log In:** Sign in with your Nexus administrator username and password.
3. **Create a Hosted Repository:**
    - Click the **gear icon** ⚙ on the top-right to access the settings.
    - In the left navigation, go to **Repositories**.
    - Click **Create repository**.
    - Select **`maven2 (hosted)`.**
    - Give the repository a name, such as **`vprofile-repo`**.



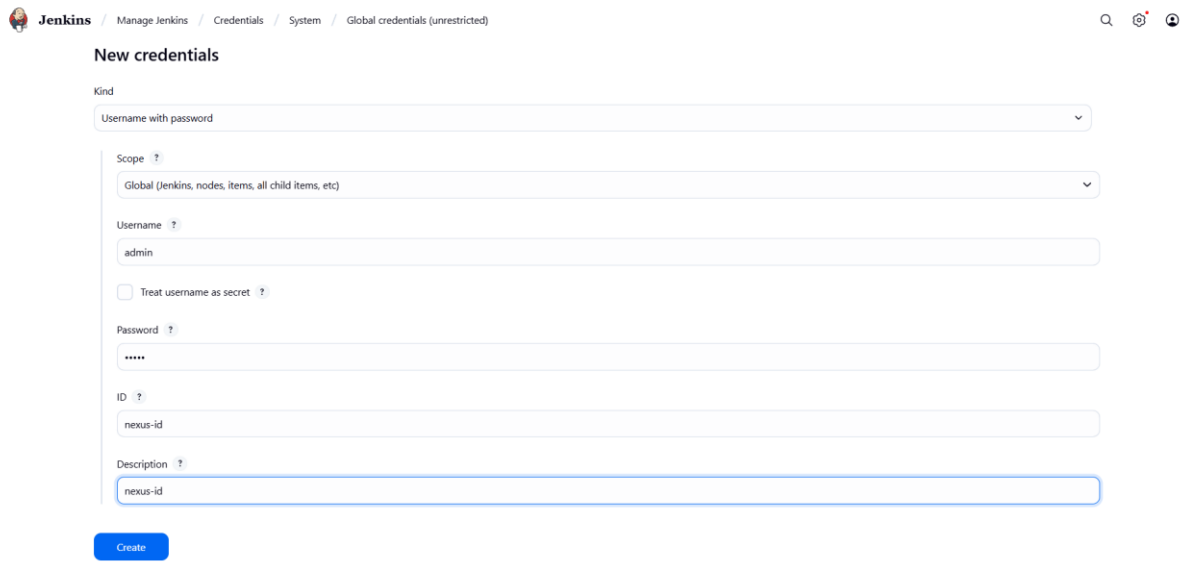
    - Leave the other settings at their defaults and click **Create repository**.

---

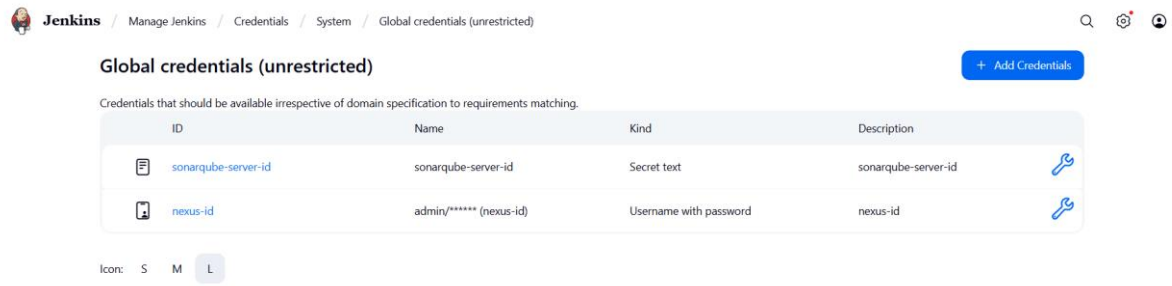
8. Integrating Nexus with Jenkins

To allow Jenkins to upload artifacts to the new Nexus repository, you must store the Nexus login credentials in Jenkins.

1. **Navigate to Credentials:** In Jenkins, go to **Manage Jenkins** -> **Manage Credentials**.
2. **Add New Credentials:**
    - Click on **`Global credentials (unrestricted)`** and then **Add Credentials**.
    - Select **Kind** as `Username with password`.
    - Enter your Nexus admin username and password.
    - Set a memorable **ID**, such as **`nexus-login`**.



- Add a description and click **Create**.



9. Uploading Artifacts to Nexus Repository

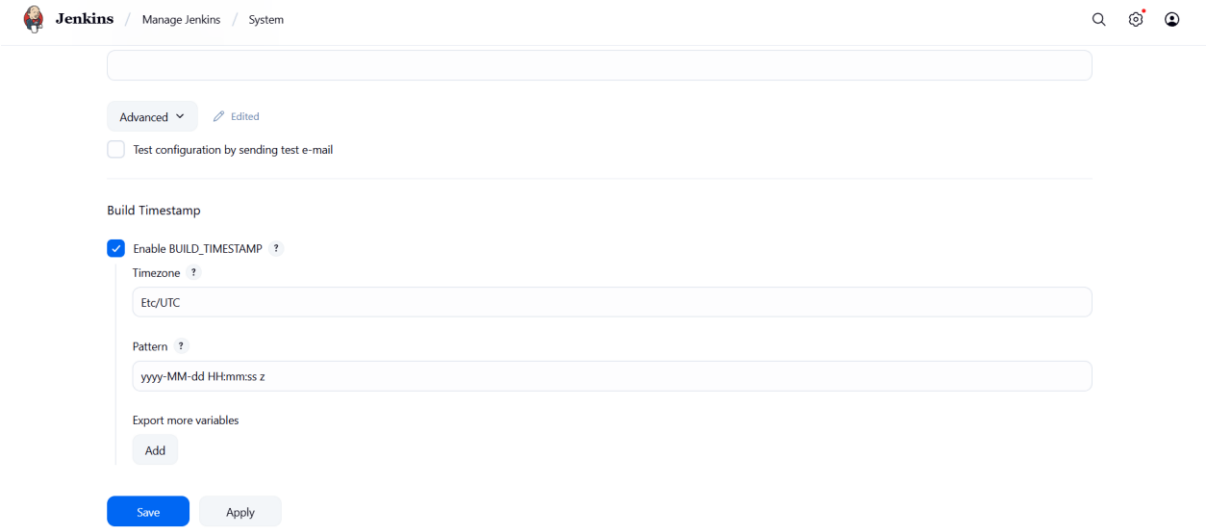
This step ensures that a tested, high-quality version of the software is stored and ready for deployment.

Modifying the Jenkinsfile

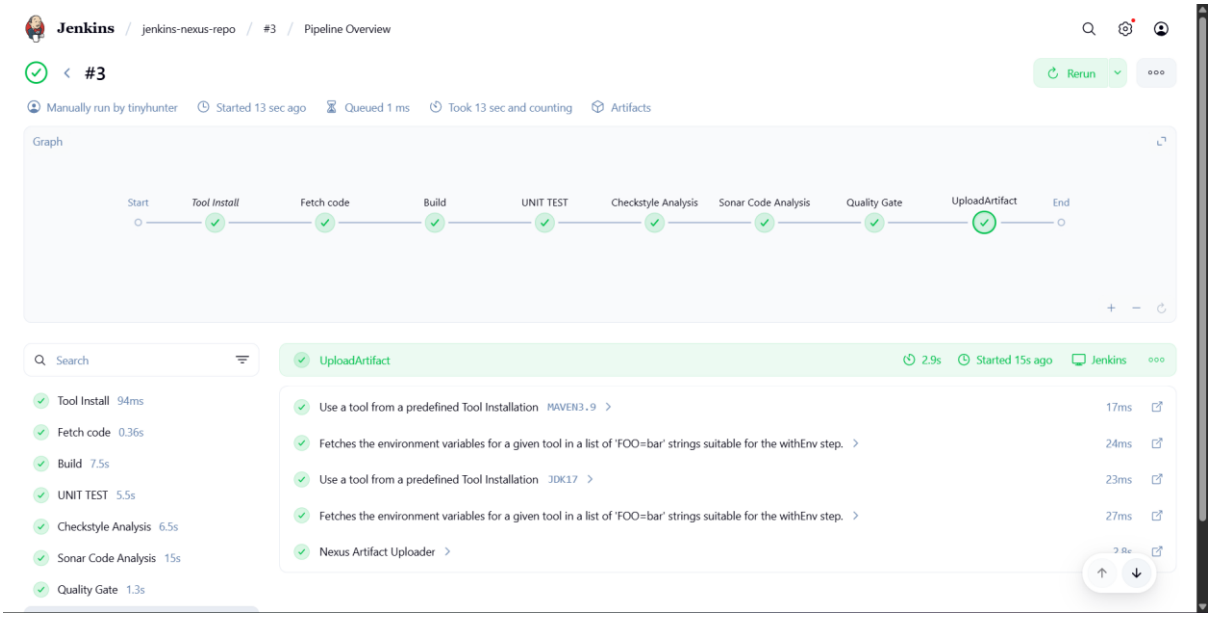
To upload the artifact, a new stage named `Upload Artifact` is added to the Jenkinsfile. This stage utilizes the **Nexus Artifact Uploader** plugin, which was installed in a previous lecture.

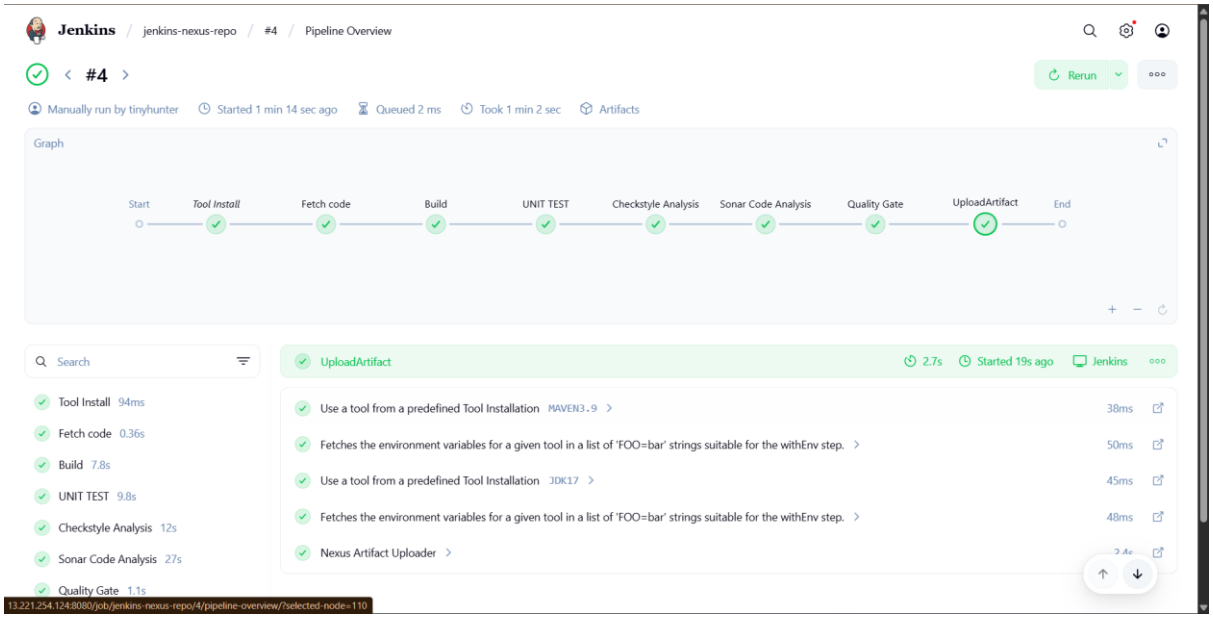
Key Elements of the `Upload Artifact` Stage:

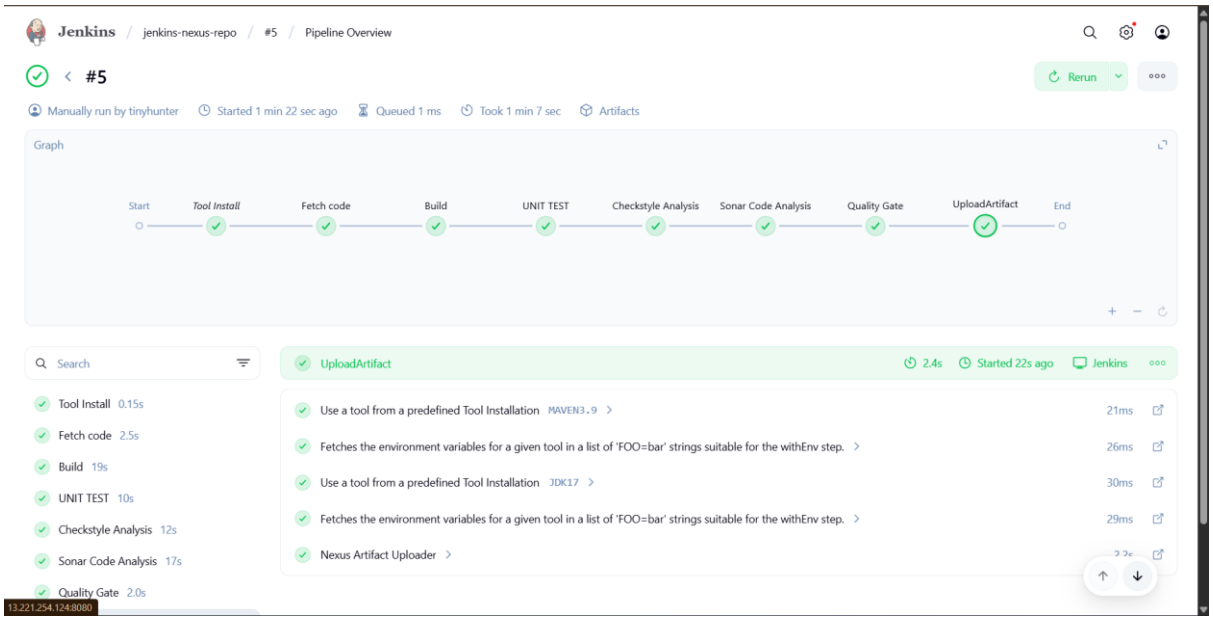
1. **Stage Definition:** A new stage block is created after the code analysis stage.
2. **Plugin Invocation:** The `nexusArtifactUploader` plugin is called with a set of parameters.
3. Now activate the timestamp: Manage Jenkins→System→Build Timestamp.

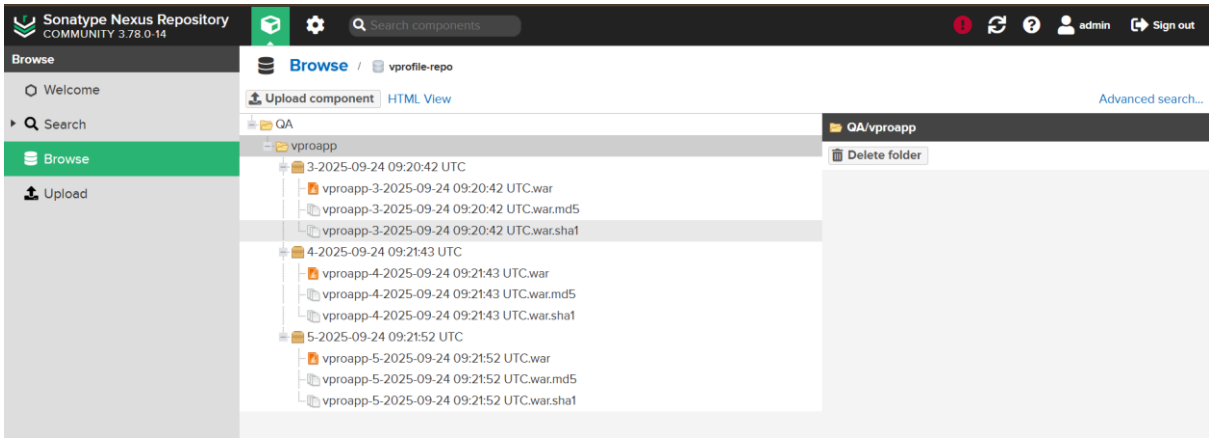


4. Here I runs continuously three builds, to check the artifacts in the nexus repo, with and build timestamp.




Jenkins / jenkins-nexus-repo / #4 / Pipeline Overview
#4
Rerun
Manually run by tinyhunter
Started 1 min 14 sec ago
Queued 2 ms
Took 1 min 2 sec
Artifacts
Graph
Start
Tool Install
Fetch code
Build
UNIT TEST
Checkstyle Analysis
Sonar Code Analysis
Quality Gate
UploadArtifact
End
Search
UploadArtifact
2.7s
Started 19s ago
Jenkins
Tool Install 94ms
Fetch code 0.36s
Build 7.8s
UNIT TEST 9.8s
Checkstyle Analysis 12s
Sonar Code Analysis 27s
Quality Gate 1.1s
Use a tool from a predefined Tool Installation MAVEN3.9
38ms
Fetches the environment variables for a given tool in a list of 'FOO=bar' strings suitable for the withEnv step.
50ms
Use a tool from a predefined Tool Installation JDK17
45ms
Fetches the environment variables for a given tool in a list of 'FOO=bar' strings suitable for the withEnv step.
48ms
Nexus Artifact Uploader
13.221.254.124:8080/job/jenkins-nexus-repo/4/pipeline-overview/?selected-node=110


Jenkins / jenkins-nexus-repo / #5 / Pipeline Overview
#5
Rerun
Manually run by tinyhunter
Started 1 min 22 sec ago
Queued 1 ms
Took 1 min 7 sec
Artifacts
Graph
Start
Tool Install
Fetch code
Build
UNIT TEST
Checkstyle Analysis
Sonar Code Analysis
Quality Gate
UploadArtifact
End
Search
UploadArtifact
2.4s
Started 22s ago
Jenkins
Tool Install 0.15s
Fetch code 2.5s
Build 19s
UNIT TEST 10s
Checkstyle Analysis 12s
Sonar Code Analysis 17s
Quality Gate 2.0s
Use a tool from a predefined Tool Installation MAVEN3.9
21ms
Fetches the environment variables for a given tool in a list of 'FOO=bar' strings suitable for the withEnv step.
26ms
Use a tool from a predefined Tool Installation JDK17
30ms
Fetches the environment variables for a given tool in a list of 'FOO=bar' strings suitable for the withEnv step.
29ms
Nexus Artifact Uploader
13.221.254.124:8080


Sonatype Nexus Repository
COMMUNITY 3.78.0-14
Search components
admin
Sign out
Browse
Welcome
Search
Browse
Upload
Browse / vprofile-repo
Upload component
HTML View
Advanced search...
QA
vproapp
3-2025-09-24 09:20:42 UTC
vproapp-3-2025-09-24 09:20:42 UTC.war
vproapp-3-2025-09-24 09:20:42 UTC.war.md5
vproapp-3-2025-09-24 09:20:42 UTC.war.sha1
4-2025-09-24 09:21:43 UTC
vproapp-4-2025-09-24 09:21:43 UTC.war
vproapp-4-2025-09-24 09:21:43 UTC.war.md5
vproapp-4-2025-09-24 09:21:43 UTC.war.sha1
5-2025-09-24 09:21:52 UTC
vproapp-5-2025-09-24 09:21:52 UTC.war
vproapp-5-2025-09-24 09:21:52 UTC.war.md5
vproapp-5-2025-09-24 09:21:52 UTC.war.sha1
QA/vproapp
Delete folder

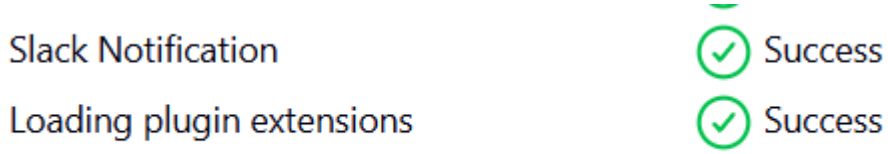
10. Setting Up Jenkins-Slack Integration

To enable Slack notifications, you need to configure both Jenkins and your Slack workspace.

1. Jenkins Plugin Installation

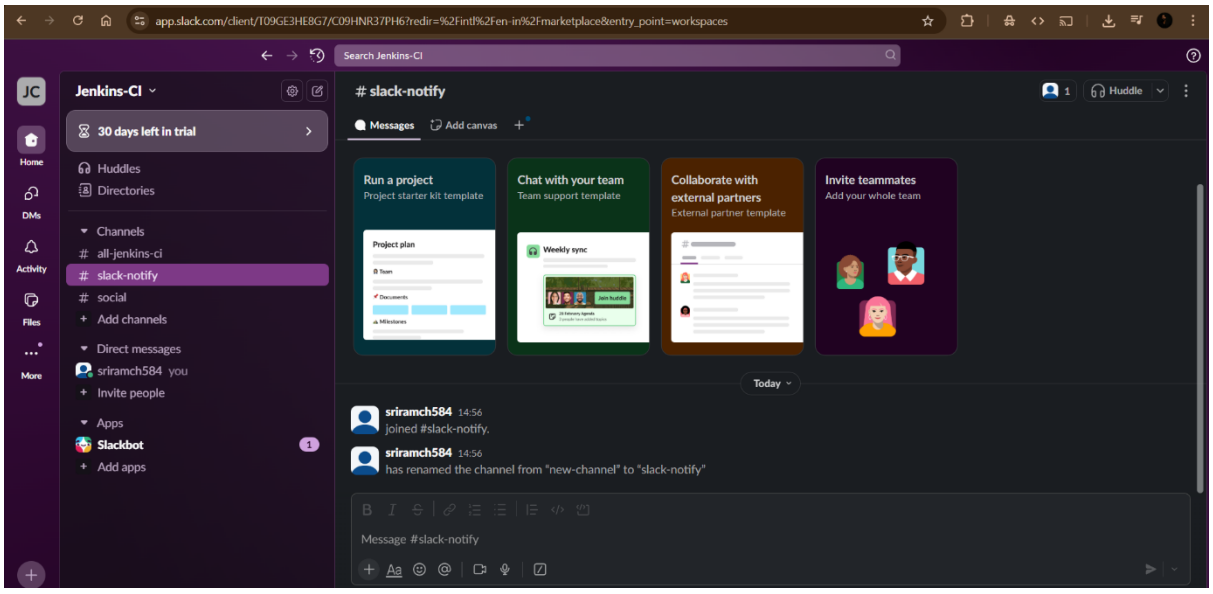
First, install the **Slack Notification Plugin** in Jenkins.

- Go to **Manage Jenkins** -> **Manage Plugins**.
- In the **Available** tab, search for `slack`.
- Select the plugin and click `Install without restart`.

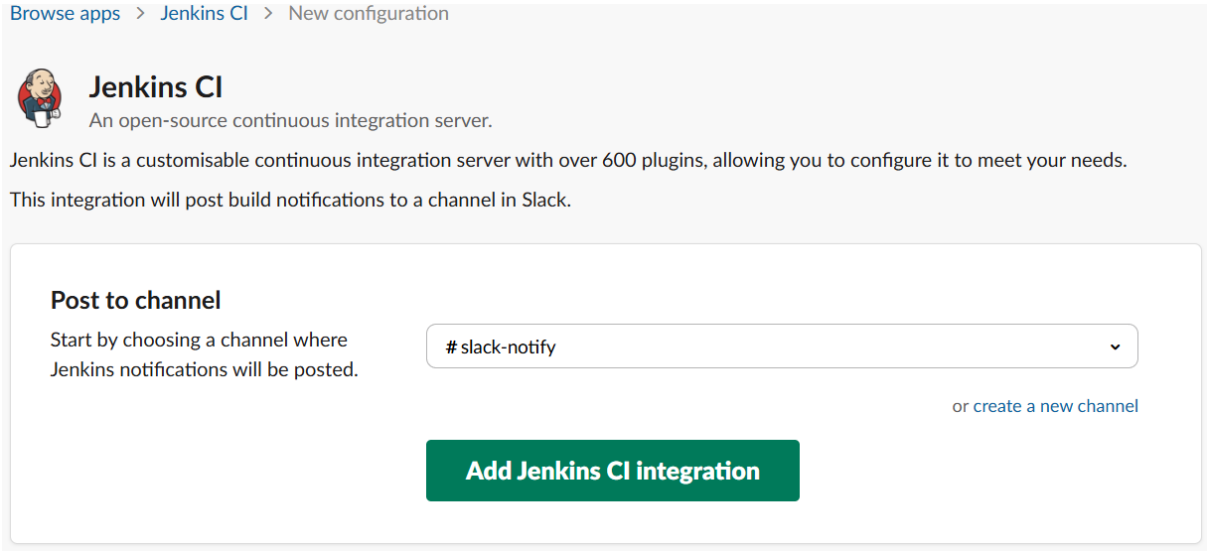


2. Slack App and Workspace Configuration

- Create a Slack account and a new workspace for your project (e.g., `Jenkins-CI`).
- In the Slack App Directory, search for and add the **Jenkins CI** app to your workspace.



- Select the channel where you want to receive notifications (e.g., `slack-notify`).

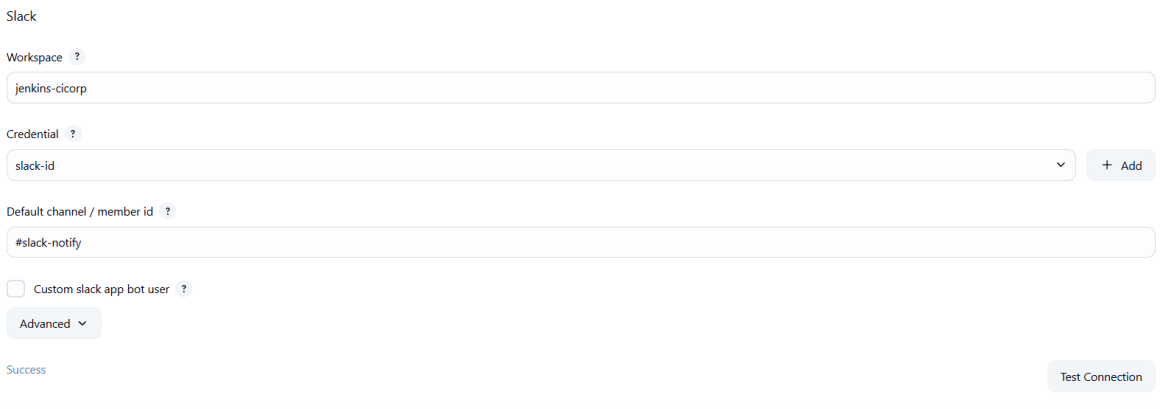


- Once the integration is added, you will be provided with a **token**. Copy this token immediately, as it's a one-time secret.

3. Jenkins System Configuration

Next, configure Jenkins to use the Slack integration details.

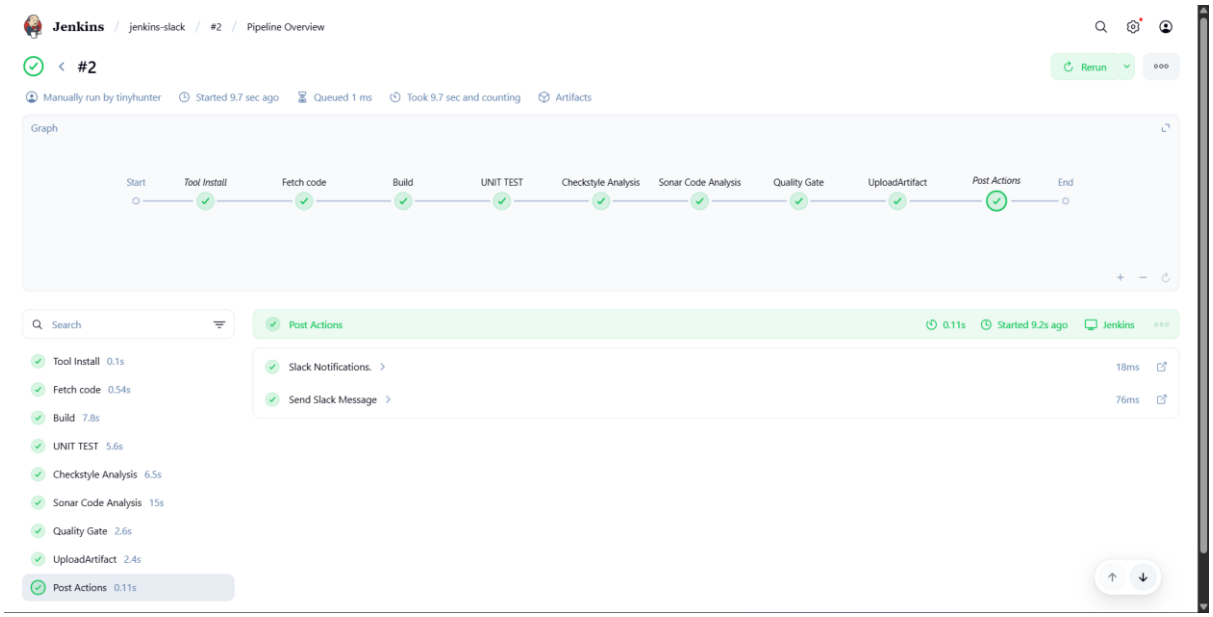
- In Jenkins, go to **Manage Jenkins** -> **Configure System**.
- Scroll down to the **Slack** section.
- Enter your **Workspace** name (e.g., `Jenkins-cicorp`). This is the portion of your Slack URL before `.slack.com`.



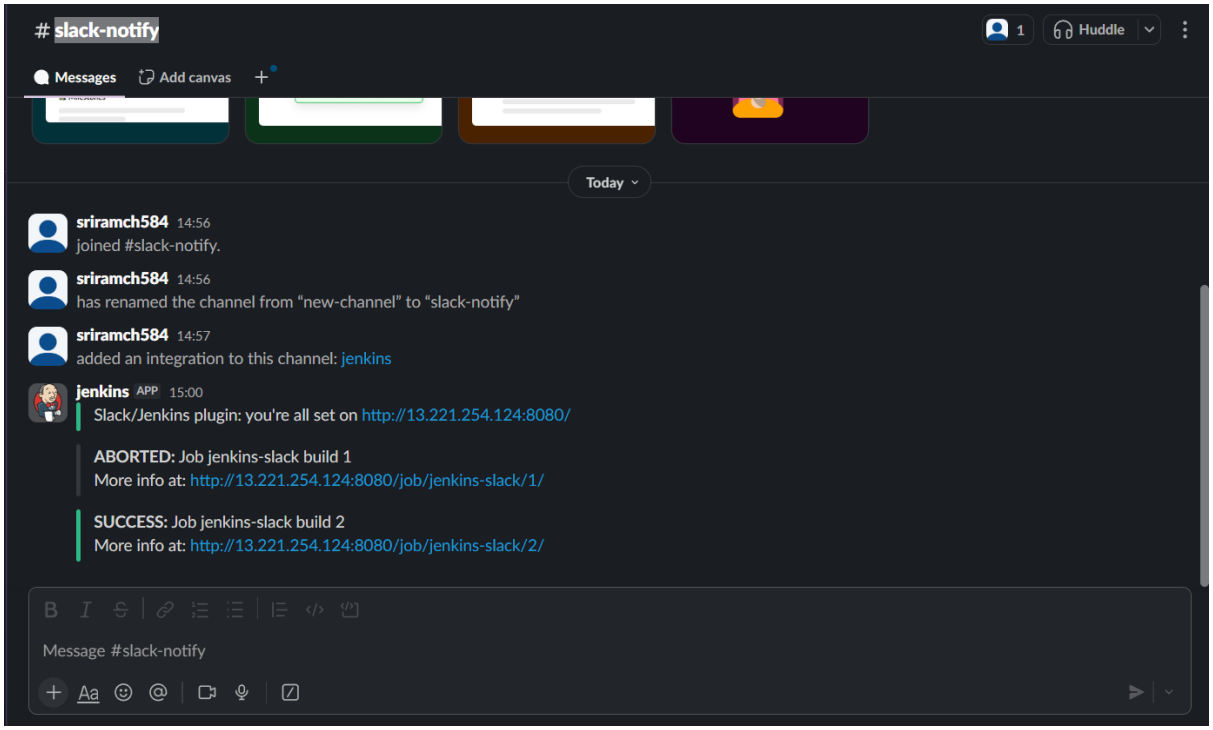
- For the **Integration Token**, click **Add** and choose `Secret text`. Paste the token you copied from Slack, give it an ID (e.g., `slack-id`), and click `Add`.
- Select the newly created credential from the dropdown.
- Enter the **Channel** name (e.g., `#slack-notify`).
- Click **Test Connection** to ensure Jenkins can successfully send messages to the Slack channel.

Adding Notifications to the Pipeline Script

Once the integration is configured, you can add a `post` section to your Jenkinsfile to handle notifications. This ensures a notification is sent regardless of whether the pipeline succeeds or fails.

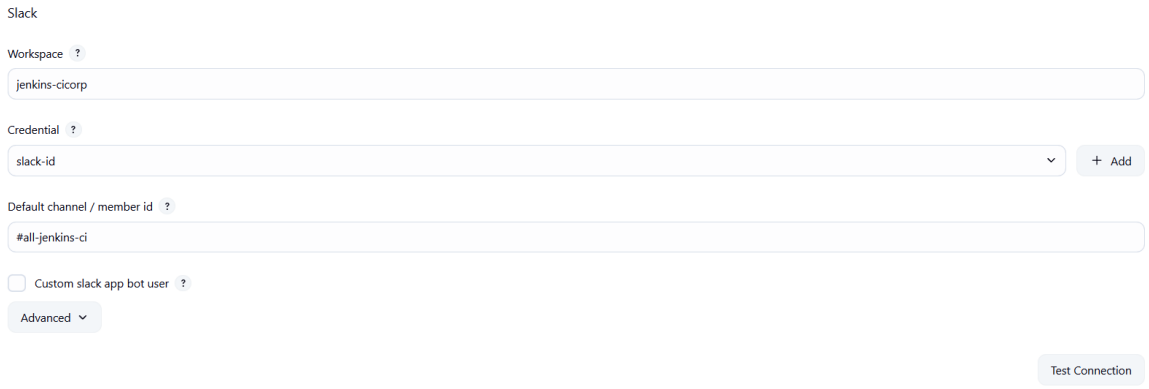


This setup provides a real-time feedback loop, alerting your team to build status directly in their collaboration channel. This helps in quickly identifying and resolving pipeline failures.

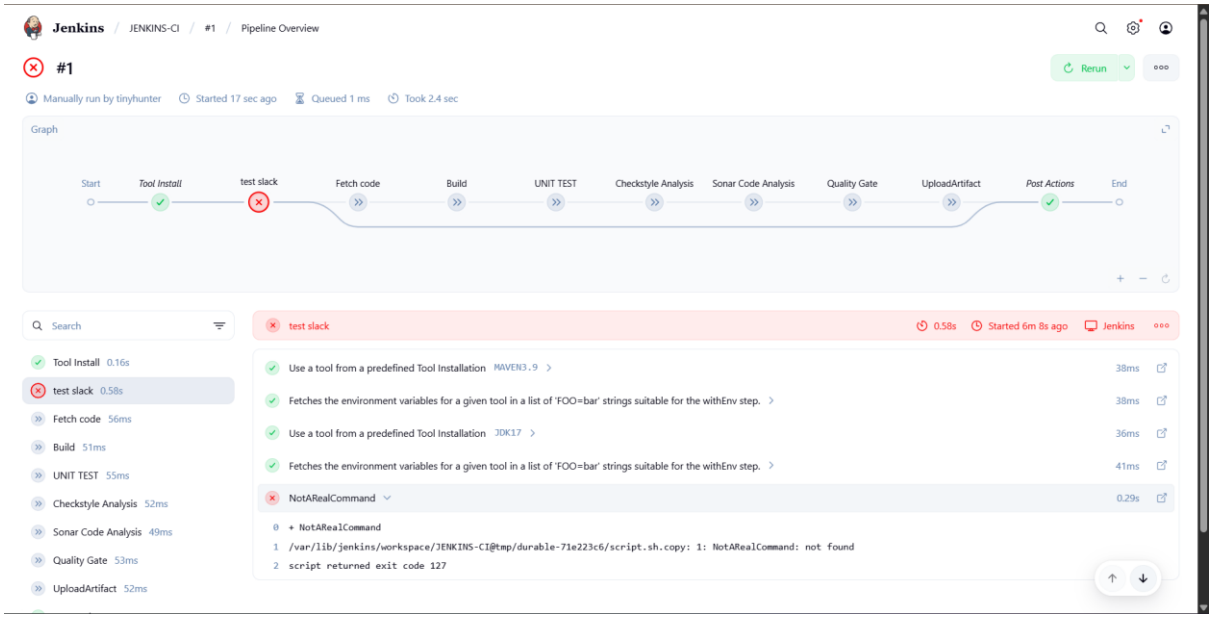


11. JENKINS CI:

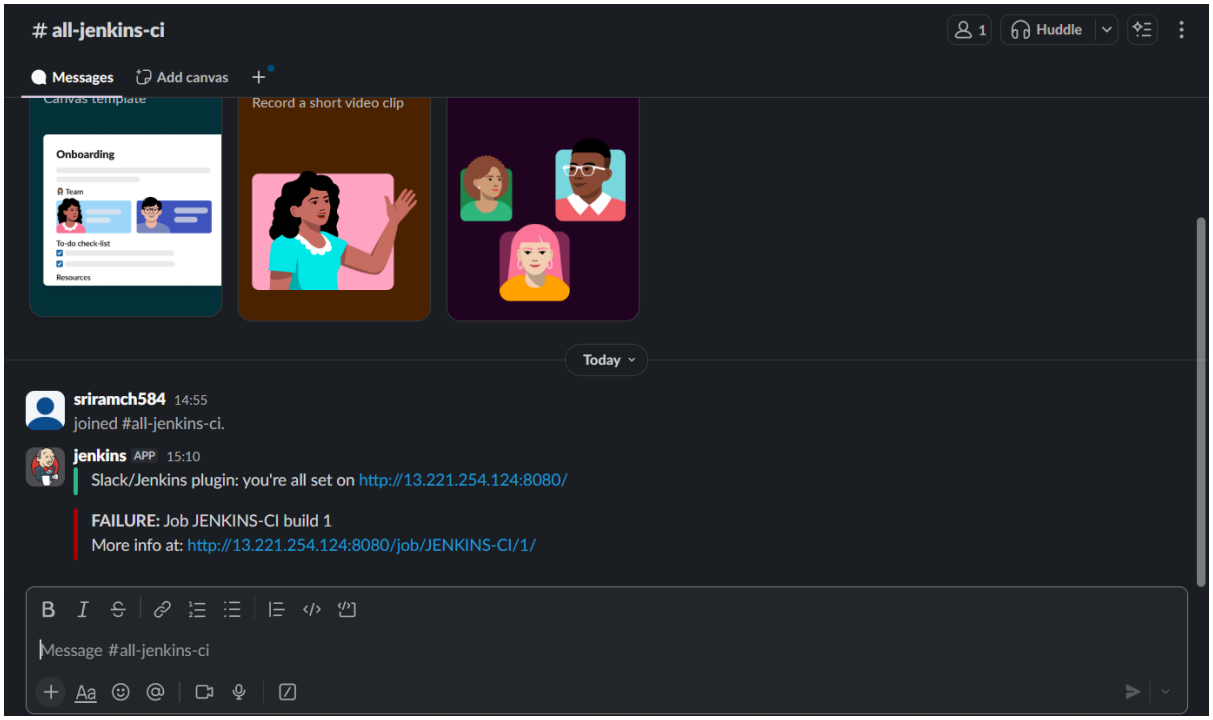
- At last we are creating the fails jobs in the pipeline, in another channel (eg:.., #all-jenkins-ci)
- For that we have to add this Configure Section as I seen in above step.



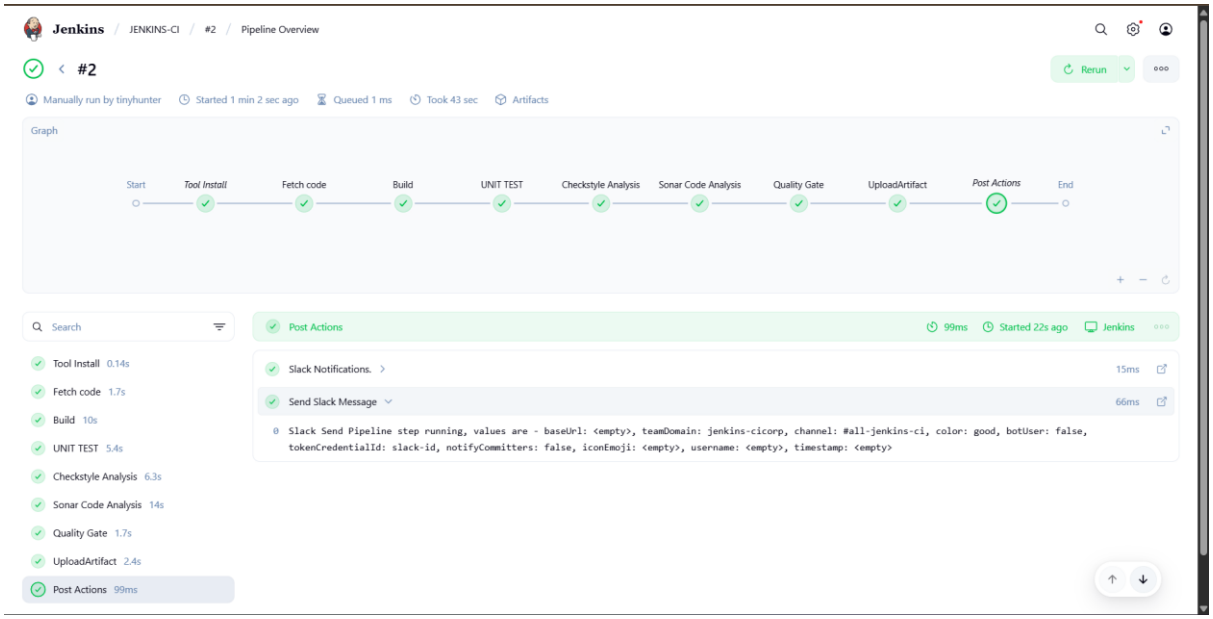
- After that run the job, by adding fake commands in the pipeline. Then it automatically fails the jobs.



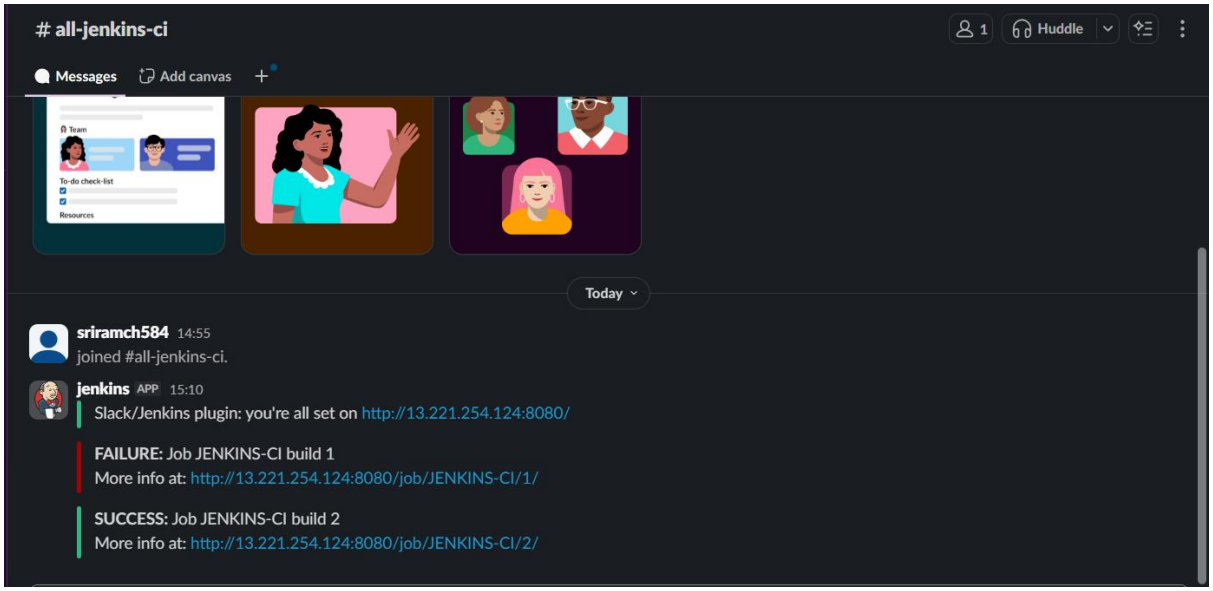
- That can be updated automatically in the slack, can be sent notification.
- Which shows the read colour is an failed job.



- Now remove the failed stage in the pipeline and run the job.



- Here we can see the slack channel has indicates the green colour, which it represents the job successful.



- Therefore this was the Jenkins CI using the pipeline.
- Here the all jobs listed.